Mục lục

Mỗi công nghệ là một mục độc lập. Nhóm chỉ cần đọc mục của công nghệ mình đăng ký.

Next.js 16 2
React 19 + Vite + TanStack 5
Angular 7
Nuxt 4 (Vue 3) 9
SvelteKit 11
Flutter 13
ASP.NET Core 15
Spring Boot 17
NestJS 20
FastAPI 22
Express.js 24
Django 26
Go + Gin 28
Go + gRPC 30
LlamaIndex (hồ sơ RAG) 32
LangGraph (hồ sơ Agentic) 35
MCP SDK (hồ sơ MCP Server) 37
Ollama (hồ sơ Local LLM) 39
Ultralytics YOLO (hồ sơ Computer Vision) 41
faster-whisper (hồ sơ Speech-to-Text) 43
Gemini API (SDK mô hình) 45
n8n 47
Power Apps + Power Automate 49
Google AppSheet (kèm hệ sinh thái Google) 51

Next.js 16

Frontend · Meta-framework trên React · Next.js 16 (Node.js 20.9+, TypeScript 5.1+, React 19.2, Turbopack là bundler mặc định)

> **Trọng tâm của đề tài**
> Next.js là framework dựng trên React, bổ sung routing theo thư mục, render phía server, lấy dữ liệu và cache. Đề tài được đánh giá ở chỗ nhóm chỉ ra được **cái gì trong đồ án là của Next.js mà một ứng dụng React thuần (SPA) không làm giống như vậy**. Giao diện đẹp hay nhiều trang không tạo thêm điểm.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Frontend (trình bày theo thuật ngữ của Next.js)

- **Component và props**: chia giao diện thành component tái sử dụng; phân biệt **Server Component** (mặc định trong App Router) và **Client Component** (khai báo `"use client"`).
- **State và data binding**: state cục bộ trong component (`useState`), state dùng chung (Context hoặc lưu trên URL bằng `searchParams`), và dữ liệu đến từ server. Với mỗi dữ liệu trong đồ án, nhóm nói được nó là loại nào và nằm ở đâu.
- **Vòng đời**: Server Component chỉ render trên server, không có `useEffect`; Client Component được hydrate rồi mới chạy effect. Giải thích hydration và nguyên nhân gây lỗi hydration mismatch.
- **Routing**: App Router, thư mục là route, `page` và `layout`, layout lồng nhau, điều hướng phía client bằng `next/link` mà không tải lại cả trang.
- **Lấy dữ liệu**: `fetch` hoặc truy vấn CSDL trực tiếp trong Server Component bất đồng bộ. Không biến mọi trang thành client component chỉ để gọi `fetch` trong `useEffect`.
- **Công cụ**: TypeScript; chạy bằng `next dev`, `next build`, `next start`; đọc được kết quả build (route nào tĩnh, route nào động). Turbopack là bundler mặc định ở bản 16.

B. Cơ chế riêng của Next.js 16

- **Ranh giới server/client**: component nào ở phía nào, vì sao; ranh giới đó ảnh hưởng tới kích thước JavaScript gửi xuống trình duyệt và dữ liệu nào bị lộ cho client.
- **API bất đồng bộ của bản 16**: `params`, `searchParams`, `cookies()`, `headers()` phải dùng với `await`. Code viết theo hướng dẫn cũ (truy cập đồng bộ) sẽ lỗi.
- **Mô hình cache mặc định**: ở bản 16, code động chạy lúc có request; cache chỉ có khi nhóm chủ động bật (Cache Components, `"use cache"`). Nhóm nói rõ đồ án của mình đang ở chế độ nào.
- **Luồng render của chính đồ án**: một request vào route cụ thể thì dữ liệu lấy ở đâu, phần nào render trên server, phần nào cần tương tác phía client.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Route động và lồng nhau; `loading.tsx`, `error.tsx`, `not-found.tsx`	Có trang danh sách và trang chi tiết	Trang chi tiết với id hợp lệ và không hợp lệ (hiện trang not-found); trạng thái loading khi dữ liệu chậm
Caching: `"use cache"`, `cacheLife`, `revalidateTag(tag, profile)`, `updateTag`	Dữ liệu ít đổi (danh mục, bài viết) hoặc cần cập nhật ngay sau khi sửa	So sánh trước và sau: số truy vấn CSDL hoặc thời gian phản hồi; tình huống dữ liệu đổi mà cache được làm mới đúng
Server Action hoặc Route Handler	Có form hoặc thao tác thay đổi dữ liệu	Chỉ rõ hàm chạy ở server; kiểm tra dữ liệu đầu vào ở server; làm mới dữ liệu sau khi ghi
Suspense và streaming	Có phần dữ liệu chậm	Vùng fallback hiện trước, dữ liệu hiện sau; ảnh hoặc video ghi lại
`next/image`, `next/font`, `generateMetadata`	Cần SEO hoặc nhiều ảnh	Cấu hình `images.remotePatterns`; kết quả Lighthouse trước và sau
`proxy.ts` (tên mới của `middleware.ts`)	Chuyển hướng hoặc kiểm tra đăng nhập đơn giản	Chỉ làm việc nhẹ ở đây; không đặt nghiệp vụ trong `proxy.ts`
Xác thực bằng cookie/session	Có đăng nhập	Route được bảo vệ ở phía server, không chỉ ẩn nút ở giao diện

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Cache Components và Partial Prerendering	Bật `cacheComponents`; tách phần tĩnh và phần động của một trang bằng Suspense; đo TTFB hoặc LCP trước/sau và giải thích vì sao khác.
React Compiler (`reactCompiler: true`)	Cài `babel-plugin-react-compiler`; đo số lần re-render bằng React DevTools Profiler trước/sau trên một màn hình có danh sách lớn; ghi lại cả thời gian build tăng lên.
Đo và giảm JavaScript phía client	Dùng bundle analyzer; chuyển một client component sang server component và báo cáo kích thước bundle giảm bao nhiêu.
Kiểm thử tự động	Component test (Vitest + Testing Library) cho một component có state; E2E bằng Playwright cho luồng chính; chạy trong CI.
Triển khai	Docker với chế độ `standalone` hoặc nền tảng hosting; quản lý biến môi trường; README để nhóm khác chạy lại được.

4. Minh chứng cần có trong báo cáo và repo

- Chỉ ra ít nhất một route thật trong đồ án và giải thích ranh giới server/client của route đó (kèm file và dòng code).
- Ảnh hoặc log chứng minh dữ liệu được lấy đúng phía đã thiết kế (ví dụ log server khi tải trang; tab Network không còn request lấy dữ liệu đó từ trình duyệt).
- Nếu dùng cache, `revalidateTag`/`updateTag`, Server Action hoặc Route Handler: có tình huống chạy trước và sau thay đổi.
- Bảng phủ kỹ thuật: mỗi mục ở phần 1 và 2 đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Tạo nhiều trang React đặt trong thư mục `app` nhưng toàn bộ đều là Client Component.
- Dùng Next.js chỉ như bộ router, còn dữ liệu vẫn lấy kiểu SPA mà không giải thích được vì sao.
- Kể tên SSR, SSG, ISR nhưng không chỉ ra được cơ chế render hay cache nào đồ án đang thật sự dùng.
- Chỉ chạy `create-next-app` rồi sửa nội dung trang mặc định.

6. Câu hỏi tự kiểm

- Cái gì trong đồ án này là Next.js mà một React SPA bình thường không làm giống như vậy?
- Nếu bỏ `"use client"` ở component này thì chuyện gì xảy ra? Vì sao?
- Dữ liệu này nên cache hay luôn lấy mới? Quyết định đó ảnh hưởng gì tới người dùng?
- Trang này render ở đâu và vào lúc nào? Nhìn vào đâu để chứng minh điều đó?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Next.js

- **React + Vite (SPA)**: đơn giản hơn, hợp ứng dụng sau đăng nhập không cần SEO; chọn Next.js khi cần render phía server, SEO, hoặc muốn giảm JavaScript gửi xuống.
- **Nuxt 4, SvelteKit, Angular (SSR)**: cùng ý tưởng ở hệ khác; chọn Next.js phải có lập luận riêng của nhóm (ví dụ hệ sinh thái React, thư viện sẵn có), không chỉ nói "phổ biến hơn".
- **Astro**: tốt cho trang nội dung tĩnh, ít tương tác; Next.js hợp hơn khi ứng dụng có nhiều trạng thái và tương tác.

Những trường hợp KHÔNG nên dùng Next.js (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Ứng dụng nội bộ sau đăng nhập, không cần SEO, ít trang: một SPA React + Vite đủ dùng và ít khái niệm hơn.
- Cần giữ kết nối thời gian thực lâu dài (WebSocket nhiều người dùng): nên có dịch vụ backend riêng; Next.js không phải nơi lý tưởng cho việc này.
- Nhóm chỉ cần một vài trang giới thiệu tĩnh: Astro hoặc HTML thuần nhẹ hơn nhiều.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một dynamic route vào module có sẵn và xử lý trường hợp không tìm thấy.
- Chuyển một thao tác cập nhật từ gọi `fetch` ở client sang Server Action hoặc Route Handler.
- Giải thích rồi chỉnh lại một ranh giới server/client chưa hợp lý trong chính đồ án.

React 19 + Vite + TanStack

Frontend · Thư viện UI + công cụ build + hệ thư viện · React 19, Vite, TanStack (Query, Router…). Công nghệ chính là cả tổ hợp; nhóm nêu rõ stack đang dùng.

> **Trọng tâm của đề tài**
> React chỉ lo giao diện. Điều làm nên một ứng dụng hoàn chỉnh là cách nhóm **chia state, lấy dữ liệu từ server, định tuyến và đóng gói**. Nếu chỉ dùng component với `useState` và `fetch` thì chưa đạt. Đề tài được đánh giá ở việc phân biệt được state phía client và dữ liệu phía server, và hiểu cơ chế cache của TanStack Query.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Frontend (trình bày theo thuật ngữ của React)

- **Component và props**: component hàm, JSX, chia nhỏ và ghép (composition); props đi một chiều từ cha xuống con.
- **State và data binding**: giao diện là hàm của state; `useState`/`useReducer` cho state cục bộ; chia sẻ state bằng nâng state lên hoặc Context; tránh lưu state dư thừa (dữ liệu suy ra được thì tính ra, không lưu).
- **Vòng đời**: mount, update, unmount thể hiện qua `useEffect` và hàm dọn dẹp (cleanup); mảng phụ thuộc làm gì; vì sao `StrictMode` ở chế độ dev chạy effect hai lần.
- **Routing phía client**: chuyển trang không tải lại toàn bộ; tham số route; route cần đăng nhập (TanStack Router hoặc router khác nhóm tự chọn và giải thích).
- **Gọi API**: gọi qua TanStack Query thay vì rải `fetch` trong `useEffect`.
- **Công cụ**: TypeScript; Vite (dev server, HMR, build production, biến môi trường `VITE_*`); chạy `npm run dev`, `build`, `preview`.

B. Cơ chế riêng của hệ React + Vite + TanStack

- **Luồng render và re-render**: state đổi thì component nào render lại; vai trò của `key` trong danh sách; khi nào cần `useMemo`/`useCallback` và khi nào không.
- **Hooks đúng quy tắc**, kèm ít nhất một custom hook do nhóm tự viết.
- **Client state và server state**: với mỗi dữ liệu trong ứng dụng, nói được nó thuộc loại nào. Dữ liệu từ backend là server state, do TanStack Query quản lý.
- **TanStack Query**: `queryKey`, `queryFn`, cache, `staleTime`/`gcTime`, trạng thái loading/error/success, và `invalidateQueries` sau mutation.
- **Vite**: vai trò của dev server và bước build, chia chunk, vì sao build production khác chạy dev.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
TanStack Router (route có kiểu, loader, search params)	Nhiều trang, có bộ lọc/phân trang lưu trên URL	Bộ lọc nằm trên URL, tải lại trang vẫn giữ nguyên; query key thay đổi theo tham số
Mutation nâng cao: optimistic update, rollback	Thao tác cần phản hồi tức thì (thích, đánh dấu hoàn thành)	Ca thành công và ca lỗi mạng (giao diện quay về trạng thái cũ)
Error Boundary và Suspense	Có màn hình dễ lỗi hoặc dữ liệu chậm	Một lỗi giả lập được chặn ở boundary, phần còn lại của ứng dụng vẫn chạy
Code splitting, `React.lazy`	Ứng dụng có nhiều trang lớn	Kích thước bundle trước/sau, thấy chunk tách riêng trong Network
Form: React Hook Form + Zod	Biểu mẫu phức tạp	Validation theo schema; thông báo lỗi đúng trường
React 19: Actions (`useActionState`), `useOptimistic`, `use`	Muốn dùng mô hình form/mutation mới	Giải thích khác gì so với cách viết cũ bằng `useState` + `onSubmit`
Quản lý client state (Context, Zustand…)	Có state UI dùng chung nhiều nơi (giỏ hàng, giao diện)	Lý do không đưa state này vào TanStack Query
Danh sách lớn: TanStack Table, Virtual	Hàng nghìn dòng	Thời gian render hoặc FPS khi cuộn trước/sau

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
React Compiler	Bật `babel-plugin-react-compiler` (hoặc plugin tương ứng của Vite); đo số lần re-render bằng React DevTools Profiler trước/sau trên một màn hình nặng; ghi lại ảnh hưởng tới thời gian build.
Hiệu năng giao diện	Profiler tìm một re-render không cần thiết và sửa; so sánh có và không virtualization với số liệu.
Cache và dữ liệu ngoại tuyến	Lưu cache của TanStack Query (persist); thử mất mạng và đồng bộ lại; nêu rõ chính sách làm mới.
Kiểm thử	Vitest + Testing Library cho component; mock API (MSW) cho test có TanStack Query; một luồng E2E bằng Playwright.
Phân tích bundle	Dùng visualizer; chỉ ra thư viện nặng nhất và cách giảm; số liệu trước/sau.
Khả năng truy cập	Kiểm tra bàn phím, focus, ARIA hoặc Lighthouse; sửa ít nhất một lỗi cụ thể và ghi số liệu.

4. Minh chứng cần có trong báo cáo và repo

- Chỉ ra một dữ liệu server state và giải thích vòng đời cache của nó (khi nào tươi, khi nào cũ, khi nào bị xóa).
- Có ít nhất một mutation làm đổi dữ liệu và cách xử lý cache sau mutation.
- Build production chạy được (`vite build` rồi `vite preview`); nếu có tối ưu bundle hoặc render thì kèm số liệu hoặc ảnh profiler.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ dùng component, `useState` và `fetch` thuần, không có phần TanStack đúng nghĩa.
- Gọi lại API sau mọi thao tác mà không hiểu cache hay invalidation.
- Dùng thư viện state hoặc query chỉ vì mẫu code AI sinh ra, không giải thích được state thuộc loại nào.
- Ghi riêng "Vite" hoặc "TanStack" làm công nghệ chính (sai loại công nghệ, bị trừ điểm).

6. Câu hỏi tự kiểm

- Dữ liệu nào trong ứng dụng là client state, dữ liệu nào là server state?
- Khi mutation thành công, vì sao query này cần invalidate hoặc refetch?
- Nếu bỏ TanStack Query, nhóm sẽ phải tự quản lý những phần nào?
- Component này render lại vì lý do gì? Em kiểm chứng bằng cách nào?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn React

- **Next.js**: có render phía server, SEO; chọn React + Vite khi ứng dụng là SPA sau đăng nhập, không cần SEO, và muốn hiểu React sâu hơn mà ít khái niệm của framework.
- **Vue/Nuxt, Angular, SvelteKit**: cùng giải bài toán giao diện; nhóm cần lập luận từ trải nghiệm làm thật.
- **Redux Toolkit/RTK Query so với TanStack Query**: RTK hợp khi có nhiều state client phức tạp; TanStack Query tập trung vào server state và cache.

Những trường hợp KHÔNG nên dùng React (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Trang nội dung chủ yếu tĩnh và cần SEO: Next.js hoặc Astro phù hợp hơn một SPA.
- Giao diện rất nhỏ chỉ vài tương tác: HTML kèm JavaScript thuần hoặc htmx là đủ, React là quá cồng kềnh.
- Đội lớn cần một cấu trúc chuẩn ép buộc từ đầu: Angular đã dựng sẵn nhiều quy ước hơn.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một mutation mới và cập nhật cache đúng cách.
- Đổi query key để hỗ trợ bộ lọc hoặc phân trang và giải thích vì sao cache tách theo tham số.
- Phân tích một re-render không cần thiết bằng Profiler rồi sửa (nếu nhóm đã làm phần hiệu năng).

Angular

Frontend · Framework đầy đủ · TypeScript · Angular 22 hoặc phiên bản Angular còn được hỗ trợ; nhóm ghi rõ phiên bản.

> **Trọng tâm của đề tài**
> Angular là framework "đủ bộ": component, dependency injection, router, form, HTTP đều có sẵn. Đề tài được đánh giá ở **kiến trúc component–service và dependency injection**, không phải ở việc dựng giao diện bằng template.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Frontend (trình bày theo thuật ngữ của Angular)

- **Component**: standalone component, `input`/`output` để giao tiếp giữa cha và con.
- **Data binding**: interpolation, property binding, event binding, two-way binding; khối điều khiển trong template (`@if`, `@for`).
- **Vòng đời**: `ngOnInit`, `ngOnDestroy` (hoặc `effect`, `afterNextRender` ở bản mới); vì sao phải hủy đăng ký (unsubscribe) khi component bị hủy.
- **State**: state cục bộ bằng signal; state dùng chung đặt trong service.
- **Routing**: Router, route và tham số, ít nhất một guard khi ứng dụng có quyền truy cập.
- **HTTP**: `HttpClient` gọi backend; xử lý lỗi; kiểu dữ liệu TypeScript cho response.
- **Công cụ**: TypeScript; Angular CLI (`ng serve`, `ng build`, `ng generate`).

B. Cơ chế riêng của Angular

- **Dependency Injection và service**: vì sao logic và dữ liệu nằm trong service thay vì dồn vào component; `inject()`; phạm vi của provider (`providedIn: 'root'` là gì).
- **Signals so với Observable (RxJS)**: signal hợp state đồng bộ của giao diện, Observable hợp luồng bất đồng bộ theo thời gian; nhóm nói rõ chỗ nào dùng cái nào.
- **Change detection**: Angular biết khi nào cập nhật giao diện bằng cách nào; chiến lược `OnPush` làm gì.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Reactive Forms với validator tự viết (đồng bộ và bất đồng bộ)	Biểu mẫu có ràng buộc nghiệp vụ	Ca hợp lệ và không hợp lệ, thông báo lỗi đúng trường
RxJS: `switchMap`, `debounceTime`, `combineLatest`…	Tìm kiếm khi gõ, kết hợp nhiều nguồn dữ liệu	Giải thích luồng observable bằng sơ đồ; không subscribe lồng nhau thiếu kiểm soát
`HttpInterceptor`	Cần gắn token, log hoặc xử lý lỗi chung	Mọi request đi qua interceptor; ca token hết hạn
Guard và resolver	Có đăng nhập, phân quyền	Đi thẳng vào URL bị chặn và chuyển hướng đúng
Lazy loading route	Có feature lớn	Chunk tách riêng trong Network; kích thước bundle ban đầu
Quản lý state bằng NgRx hoặc signal store	State dùng chung phức tạp	Luồng action – reducer/selector hoặc signal store cho một chức năng
`@defer` (tải hoãn phần giao diện)	Phần giao diện nặng, ít khi dùng	Thấy phần được tải hoãn trong Network
Angular Material hoặc CDK	Cần thành phần giao diện dựng sẵn	Dùng đúng CDK/Material, không bê nguyên mẫu mà không hiểu

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Hiệu năng change detection	Chuyển `OnPush` hoặc zoneless; đo số chu kỳ change detection bằng Angular DevTools trước/sau trên một danh sách lớn.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
SSR và hydration	Bật server-side rendering; đo LCP hoặc TTFB trước/sau và giải thích hydration.
Kiểm thử	Unit test component/service bằng `TestBed`; E2E bằng Playwright; chạy trong CI.
Phân tích bundle	Dùng source-map-explorer hoặc công cụ tương tự; đặt performance budget và chứng minh build vượt ngưỡng thì báo lỗi.
Khả năng truy cập	Kiểm tra bàn phím và ARIA với CDK; Lighthouse trước/sau.

4. Minh chứng cần có trong báo cáo và repo

- Cây component và service rõ ràng; chỉ ra service được inject ở đâu.
- Route thật và guard hoạt động (nếu đã chọn), kèm ảnh ca được vào và ca bị chặn.
- Nếu dùng RxJS hoặc signals: ví dụ luồng dữ liệu trong code, không chỉ nhập thư viện.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Đặt toàn bộ logic gọi API và xử lý dữ liệu trong một component lớn.
- Dùng service như lớp "cho có" nhưng component vẫn phụ thuộc trực tiếp vào mọi thứ.
- Dùng rất nhiều toán tử RxJS nhưng không giải thích được luồng observable.

6. Câu hỏi tự kiểm

- Vì sao chỗ này dùng service chứ không đặt logic thẳng trong component?
- Guard của em bảo vệ điều gì? Nếu người dùng đi thẳng vào URL thì sao?
- State này phù hợp signal hay observable hơn? Vì sao?
- Component này được kiểm tra thay đổi (change detection) khi nào?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Angular

- **React + Vite, Vue/Nuxt**: nhẹ và linh hoạt hơn, ít quy ước; chọn Angular khi cần cấu trúc chặt, DI chuẩn và đội nhiều người.
- **Next.js**: có SSR sẵn; Angular cũng có SSR nhưng nhóm cần so sánh bằng trải nghiệm thật.

Những trường hợp KHÔNG nên dùng Angular (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Trang giới thiệu hoặc ứng dụng nhỏ vài màn hình: Angular nặng và nhiều khái niệm so với nhu cầu.
- Nhóm chưa quen TypeScript và lập trình hướng đối tượng: thời gian sẽ dành cho học ngôn ngữ và khái niệm DI thay vì làm chủ công nghệ.
- Cần dựng prototype rất nhanh: framework ít quy ước hơn đạt kết quả sớm hơn.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một route có guard và thông báo điều hướng phù hợp.
- Tách một khối logic lớn khỏi component sang service và giải thích dependency.
- Thêm validation bất đồng bộ hoặc xử lý lỗi HTTP tập trung.

Nuxt 4 (Vue 3)

Frontend · Meta-framework trên Vue 3 · Nuxt 4. Không đăng ký Nuxt 3 vì đã hết vòng đời hỗ trợ từ 31/7/2026.

> **Trọng tâm của đề tài**
> Nuxt là Vue cộng thêm routing theo tệp, render phía server (SSR), hydration và cơ chế lấy dữ liệu của Nuxt. Đề tài được đánh giá ở chỗ nhóm chỉ ra được **cái gì là Nuxt mà một Vue SPA thông thường không làm**. Một ứng dụng Vue + Vite không kèm Nuxt cần trao đổi riêng với giảng viên.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Frontend (trình bày theo thuật ngữ của Vue/Nuxt)

- **Component**: Single File Component với `<script setup>`; props và emit; slot.
- **Data binding và phản ứng**: `ref`, `reactive`, `computed`, `watch`; `v-model`; giao diện tự cập nhật khi dữ liệu đổi.
- **Vòng đời**: `onMounted`, `onUnmounted`; trong Nuxt, code chạy ở server trước rồi mới hydrate ở client, nên API của trình duyệt chỉ dùng được ở phía client.
- **State**: state cục bộ bằng `ref`; state dùng chung bằng `useState` của Nuxt hoặc Pinia khi ứng dụng thật sự có state chung.
- **Routing**: tệp trong thư mục `pages/` tự thành route; route động (`[id].vue`); layout; điều hướng bằng `NuxtLink`.
- **Lấy dữ liệu**: `useFetch` hoặc `useAsyncData`; xử lý rõ trạng thái pending, error, data.
- **Công cụ**: TypeScript; Nuxt CLI; auto-import; cấu trúc thư mục mặc định của Nuxt 4 (thư mục `app/`).

B. Cơ chế riêng của Nuxt

- **SSR và hydration trong chính ứng dụng**: phần nào render trên server, phần nào được client tiếp quản sau hydration; vì sao có lỗi hydration mismatch.
- **Lấy dữ liệu theo cách của Nuxt**: vì sao dùng `useFetch`/`useAsyncData` thay cho `onMounted` + `fetch` (tránh gọi hai lần, dữ liệu theo payload từ server).
- **Cấu trúc theo quy ước**: route sinh từ cấu trúc tệp; nhóm chỉ được một route thật và đường đi dữ liệu của route đó.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Composable tự viết	Một logic dùng lại ở nhiều trang	Composable được dùng ở ít nhất hai trang
Route middleware	Cần chuyển hướng hoặc chặn theo đăng nhập	Ca bị chặn và ca được vào; khác gì server middleware
Server route của Nitro (`server/api`)	Cần endpoint ngay trong dự án	Gọi endpoint từ trang; giải thích chạy ở server
`routeRules`, caching theo route	Có trang tĩnh, trang cần làm mới định kỳ	Hai route dùng hai chế độ render khác nhau và kết quả đo
Pinia hoặc `useState`	State dùng chung nhiều trang	Lý do chọn công cụ nào; nhận biết state được giữ giữa server và client
SEO: `useHead`, `useSeoMeta`	Cần chia sẻ và công cụ tìm kiếm	Thẻ meta đúng cho từng trang động
Error page và xử lý lỗi	Muốn trang 404 và lỗi thân thiện	`error.vue`, ca id không tồn tại
Module Nuxt (Image, i18n, UI…)	Có nhu cầu tương ứng	Nêu module nào làm gì, cấu hình ra sao

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Hybrid rendering theo route	Dùng `routeRules` để trộn SSR, prerender, SWR; đo TTFB hoặc LCP giữa các chế độ và giải thích khác biệt.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Cache ở tầng Nitro	Cache một API ở server; đo thời gian phản hồi và số lần chạm CSDL trước/sau.
Chi phí hydration và kích thước payload	Đo kích thước payload và JavaScript phía client; giảm bằng cách chuyển hoặc lazy hóa component.
Kiểm thử	`@nuxt/test-utils` với Vitest cho component và composable; một luồng E2E.
Triển khai	Docker hoặc preset Node; quản lý biến môi trường; README để nhóm khác chạy lại.

4. Minh chứng cần có trong báo cáo và repo

- Ít nhất một trang lấy dữ liệu theo cơ chế Nuxt và giải thích SSR/hydration của trang đó.
- Một route sinh từ cấu trúc tệp và đường đi dữ liệu của route.
- Nếu dùng server route, middleware hoặc composable: chỉ ra nơi được gọi và lý do dùng.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Dùng Nuxt như một Vue SPA thông thường nhưng không giải thích được SSR/hydration.
- Dùng `onMounted` + `fetch` cho mọi trang dù có thể lấy dữ liệu bằng cơ chế Nuxt.
- Kể tên Nitro hoặc middleware nhưng đồ án không có ví dụ thật.

6. Câu hỏi tự kiểm

- Trang này render gì ở server? Sau hydration điều gì thay đổi?
- Vì sao dùng `useFetch`/`useAsyncData` ở đây thay vì `onMounted`?
- Route middleware này khác server middleware ở điểm nào trong đồ án?
- Nếu bỏ Nuxt và dùng Vue + Vite, em mất những gì?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Nuxt

- **Vue + Vite (SPA)**: đơn giản hơn khi không cần SEO hay SSR; nhưng danh mục yêu cầu trao đổi riêng nếu đăng ký như vậy.
- **Next.js, SvelteKit**: cùng ý tưởng meta-framework ở hệ khác; Nuxt có cú pháp dễ vào nhất trong ba hệ lớn nhưng nhóm vẫn cần lập luận riêng.

Những trường hợp KHÔNG nên dùng Nuxt (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Ứng dụng nội bộ sau đăng nhập không cần SEO: Vue + Vite đủ dùng, không cần chi phí SSR.
- Nhóm cần hệ thư viện và nguồn tài liệu lớn nhất cho giao diện: hệ React lớn hơn.
- Cần giữ kết nối thời gian thực lâu dài với nhiều người dùng: nên có dịch vụ backend riêng.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một dynamic page dùng `useAsyncData` và xử lý lỗi.
- Viết một composable dùng lại ở hai trang hiện có.
- Chuyển một API đơn giản vào Nitro server route nếu kiến trúc đồ án đã có phần server.

SvelteKit

Frontend · Meta-framework trên Svelte · Svelte 5 (runes) và SvelteKit. Nhóm ghi rõ phiên bản đang dùng.

> **Trọng tâm của đề tài**
> Svelte biên dịch component thành mã cập nhật giao diện trực tiếp (không có virtual DOM), SvelteKit thêm routing, hàm `load` và ranh giới server–client. Đề tài được đánh giá ở chỗ nhóm hiểu **mô hình reactivity của Svelte và nơi code chạy (server hay client)**. Chọn SvelteKit vì "ít người dùng nên khác biệt" không phải là lập luận kỹ thuật.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Frontend (trình bày theo thuật ngữ của Svelte/SvelteKit)

- **Component**: tệp `.svelte` gồm script, markup và style; props; sự kiện và snippet.
- **Data binding và phản ứng**: runes `$state`, `$derived`, `$effect`; `bind:`; giao diện cập nhật khi state đổi.
- **Vòng đời**: `onMount` và dọn dẹp trong `$effect`; code chỉ chạy ở trình duyệt đặt ở đâu.
- **State**: state cục bộ trong component; state dùng chung bằng store hoặc context khi thật sự cần.
- **Routing**: thư mục `src/routes`, `+page.svelte`, `+layout.svelte`, route động `[slug]`; điều hướng phía client.
- **Lấy dữ liệu**: hàm `load` trong `+page.ts` hoặc `+page.server.ts` và cách dữ liệu đi vào trang.
- **Công cụ**: TypeScript; Vite; tạo dự án bằng CLI chính thức; adapter để build.

B. Cơ chế riêng của SvelteKit

- **Reactivity của Svelte**: biên dịch ra mã cập nhật trực tiếp thay vì so sánh cây ảo; điều đó khác React thế nào ở mức sử dụng.
- **Ranh giới server–client**: `+page.server.ts` chỉ chạy ở server và `+page.ts` chạy cả hai phía; không dùng API trình duyệt ở server; không để secret lộ sang client (`$env/static/private`).
- **Hàm `load`**: chạy ở đâu, khi nào; dữ liệu trả về được tuần tự hóa ra sao.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Form actions và `use:enhance`	Có form gửi dữ liệu	Form hoạt động cả khi tắt JavaScript; so sánh với `fetch` thủ công
SSR hoặc prerender theo loại trang	Có trang tĩnh và trang động	Hai trang dùng hai chế độ khác nhau, giải thích vì sao
`hooks.server.ts`	Xử lý chung như xác thực, ngữ cảnh	Request được gắn thông tin người dùng; ca chưa đăng nhập
Stores hoặc context	State dùng chung	Lý do chọn store hay context; không dùng biến toàn cục ở server
Streaming dữ liệu chậm trong `load`	Có dữ liệu chậm	Phần nhanh hiện trước, dữ liệu chậm hiện sau
API route `+server.ts`	Cần endpoint trong dự án	Gọi và kiểm tra bằng request mẫu
Validation form (Zod, Superforms)	Biểu mẫu phức tạp	Lỗi hiển thị đúng trường, kiểm tra cả ở server
Adapter và triển khai	Muốn đào sâu build/deploy	Chạy được với adapter đã chọn; README

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
So sánh kích thước và hiệu năng	Dựng cùng một màn hình ở Svelte và một framework khác (hoặc đo thật phần đã làm); báo cáo kích thước JavaScript và thời gian tương tác.
Progressive enhancement	Chứng minh một luồng chính (đăng nhập, gửi form) vẫn chạy khi tắt JavaScript, rồi nâng cấp bằng `use:enhance`.
Kiểm thử	Vitest cho logic; Playwright cho luồng chính; chạy trong CI.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Hiệu năng tải trang	Lighthouse hoặc Web Vitals trước/sau khi đổi chế độ render hoặc tách dữ liệu chậm.
Khả năng truy cập	Kiểm tra bàn phím, focus, ARIA; sửa cảnh báo của trình biên dịch Svelte (a11y).

4. Minh chứng cần có trong báo cáo và repo

- Ít nhất một route dùng `load` và một tương tác làm đổi state hoặc dữ liệu.
- Chỉ rõ file hoặc module nào chỉ chạy ở server và lý do.
- Nếu dùng form action, hook hoặc store: luồng sử dụng thật trong ứng dụng.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ viết component Svelte nhưng không dùng cơ chế của SvelteKit.
- Đưa secret hoặc API key vào code client.
- Không phân biệt được dữ liệu nào được load ở server và dữ liệu nào sinh ở trình duyệt.

6. Câu hỏi tự kiểm

- Hàm `load` này chạy ở đâu và khi nào?
- Nếu đoạn code này dùng `window` hoặc `localStorage` thì có chạy được ở server không?
- Form action giải quyết vấn đề gì so với `fetch` thủ công?
- `$state` khác `useState` của React ở điểm nào?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn SvelteKit

- **Next.js, Nuxt**: hệ sinh thái và việc làm nhiều hơn; chọn SvelteKit vì cú pháp ngắn, ít mã mẫu, bundle nhỏ, và nhóm cần có lập luận kỹ thuật cụ thể.
- **Astro**: hợp trang nội dung; SvelteKit hợp ứng dụng có nhiều tương tác hơn.

Những trường hợp KHÔNG nên dùng SvelteKit (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Phụ thuộc nhiều vào thư viện giao diện chỉ có bản cho React: hệ sinh thái Svelte nhỏ hơn.
- Nhóm hoặc doanh nghiệp mục tiêu cần nhiều nhân sự quen framework: Svelte ít phổ biến hơn.
- Ứng dụng cần mô hình component thật sự khác (ví dụ cần React Native cho mobile cùng mã): chọn hệ sinh thái tương ứng.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một form action vào một luồng có sẵn.
- Chuyển một trang tĩnh sang prerender hoặc SSR phù hợp và giải thích.
- Sửa một lỗi do dùng API chỉ có ở trình duyệt trên sai phía (server).

Flutter

Cross-platform · UI framework và SDK · Dart 3 · Flutter (phiên bản ổn định mới); nhóm ghi rõ nền tảng đích (Android, iOS, web, desktop).

> **Trọng tâm của đề tài**
> Flutter vẽ giao diện bằng cây widget khai báo và tự quản lý việc dựng lại khi state đổi. Đề tài được đánh giá ở chỗ nhóm hiểu **cây widget, cơ chế rebuild và quản lý trạng thái**, không phải ở số màn hình hay độ đẹp.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng (trình bày theo thuật ngữ của Flutter)

- **Widget và composition**: mọi thứ là widget; `StatelessWidget` và `StatefulWidget`; ghép widget nhỏ thành màn hình; layout (Row, Column, Stack, Expanded, ListView).
- **State và cơ chế rebuild**: khi trạng thái đổi, `build()` được gọi lại cho widget nào; `setState` làm gì; vai trò của `const` và `Key`. Nhóm chứng minh được widget nào được dựng lại (bằng DevTools hoặc log).
- **Vòng đời**: `initState`, `didUpdateWidget`, `dispose`; giải phóng controller, stream subscription.
- **Quản lý trạng thái cấp ứng dụng**: chọn một giải pháp (Provider, Riverpod, Bloc hoặc tương đương) và giải thích vì sao; tách logic ra khỏi widget.
- **Điều hướng**: Navigator hoặc `go_router`, truyền tham số giữa màn hình, màn hình cần đăng nhập.
- **Bất đồng bộ**: `Future`, `async/await`, `Stream`; `FutureBuilder`/`StreamBuilder` hoặc tương đương.
- **Công cụ**: `pubspec.yaml`, package, `flutter doctor`, hot reload, DevTools; chạy được trên ít nhất một nền tảng đích.

B. Cơ chế riêng của Flutter

- **Ba cây**: widget, element, render object ở mức giải thích được vì sao widget "rẻ" còn render object thì "đắt".
- **Giao diện khai báo**: giao diện là hàm của state; so sánh ngắn với cách dựng giao diện bằng XML/View.
- **Chạy đa nền tảng**: cùng một mã chạy trên nhiều nền tảng, những phần khác nhau theo nền tảng (quyền, plugin) nằm ở đâu.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Gọi API: `http` hoặc `dio`, tuần tự hóa JSON thành model	Có backend	Model có kiểu; xử lý lỗi mạng và JSON sai định dạng
Lưu trữ cục bộ: `shared_preferences`, SQLite (drift/sqflite), Hive	Cần dữ liệu ngoại tuyến hoặc lưu cấu hình	Tắt mạng vẫn xem được dữ liệu đã lưu
Plugin và platform channel	Cần camera, định vị, thông báo…	Quyền được xin đúng; xử lý khi bị từ chối
Theming (Material 3), responsive/adaptive	Chạy trên nhiều kích thước màn hình	Ảnh hai kích thước màn hình khác nhau
Hoạt họa (implicit/explicit)	Cần chuyển động có mục đích	Hoạt họa gắn với state, không chỉ trang trí
Kiểm thử widget và integration test	Luôn nên có	Test widget cho một màn hình có state; kết quả chạy
Firebase (xác thực, dữ liệu) như dịch vụ hỗ trợ	Cần backend nhanh	Danh mục xếp Firebase là công nghệ hỗ trợ, không phải công nghệ chính

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Đo hiệu năng bằng DevTools	Đo thời gian dựng khung hình (mục tiêu giữ dưới ~16 ms ở 60 FPS) trên một màn hình có danh sách dài; tìm và sửa một widget rebuild thừa (dùng `const`, tách widget); số liệu trước/sau.
Tính toán nặng ngoài luồng giao diện	Dùng `Isolate`/`compute` cho tác vụ nặng; so sánh độ giật màn hình trước/sau.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Ngoại tuyến và đồng bộ	Lưu cục bộ trước, đồng bộ khi có mạng; xử lý xung đột ở mức đơn giản.
Kiểm thử nâng cao	Golden test cho giao diện; integration test chạy trên thiết bị hoặc giả lập; chạy trong CI.
Phát hành	Build APK/AAB hoặc bản web; cấu hình ký ứng dụng, biến môi trường theo môi trường.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ cây widget của một màn hình thật và chỉ ra widget nào dựng lại khi state đổi (ảnh DevTools hoặc log).
- Giải pháp quản lý trạng thái đã chọn và luồng dữ liệu của một chức năng.
- Ảnh chụp hoặc quay màn hình chạy trên ít nhất một nền tảng, kèm kết quả `flutter doctor`.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Dựng nhiều màn hình bằng `StatefulWidget` và `setState` ở mọi nơi, không tách logic và không giải thích được rebuild.
- Gọi API trực tiếp trong `build()` khiến mỗi lần dựng lại lại gọi mạng.
- Không giải phóng controller hoặc subscription, gây rò rỉ.

6. Câu hỏi tự kiểm

- Khi trạng thái đổi, những widget nào được dựng lại? Em kiểm chứng bằng gì?
- `const` widget giúp gì? Khi nào `Key` là cần thiết?
- Vì sao em chọn giải pháp quản lý trạng thái này thay vì `setState`?
- `dispose()` ở đây giải phóng cái gì? Bỏ đi thì sao?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Flutter

- **React Native + Expo**: tận dụng kiến thức React/JavaScript, Expo giúp thực hành nhẹ; Flutter cho kiểm soát giao diện nhất quán giữa các nền tảng vì tự vẽ.
- **Android native (Kotlin + Jetpack Compose)**: chạy tốt nhất trên Android và dùng API hệ thống đầy đủ; Flutter đáng chọn khi cần nhiều nền tảng từ một mã nguồn.

Những trường hợp KHÔNG nên dùng Flutter (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Chỉ nhắm một nền tảng và cần dùng sâu API riêng của nền tảng đó: native hợp hơn.
- Ứng dụng chủ yếu là nội dung web cần SEO: Flutter web không mạnh về SEO.
- Nhóm cần giữ kích thước ứng dụng rất nhỏ: gói Flutter có kích thước cơ sở lớn hơn.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Đổi một `setState` rải rác sang giải pháp quản lý trạng thái đã chọn cho một chức năng.
- Tìm một widget rebuild thừa trong màn hình hiện có và sửa.
- Thêm xử lý lỗi mạng cho một luồng gọi API.

ASP.NET Core

Backend · Framework C# · ASP.NET Core trên .NET 8 LTS trở lên (hoặc .NET 10 LTS); nhóm ghi rõ phiên bản.

> **Trọng tâm của đề tài**
> Điểm đặc trưng của ASP.NET Core là **middleware pipeline, dependency injection tích hợp sẵn và cách tổ chức endpoint**. Đề tài được đánh giá ở chỗ nhóm hiểu thứ tự xử lý request và vòng đời service, không phải ở số bảng CRUD.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Backend (trình bày theo thuật ngữ của ASP.NET Core)

- **Luồng một request**: request đi qua pipeline, đến endpoint (Controller hoặc Minimal API), đến service, đến EF Core, rồi trả JSON. Vẽ lại luồng này cho một chức năng thật.
- **Routing và model binding**: attribute routing hoặc Minimal API; binding từ route, query, body; validation (Data Annotations hoặc FluentValidation).
- **Truy cập dữ liệu**: EF Core, `DbContext`, entity và quan hệ, migration; biết nhận ra vấn đề N+1.
- **Xử lý lỗi nhất quán**: mã HTTP đúng, `ProblemDetails`, middleware xử lý ngoại lệ.
- **Cấu hình**: `appsettings.json` theo môi trường, user-secrets, biến môi trường; không commit bí mật.
- **Công cụ**: `dotnet` CLI, NuGet, cấu trúc `Program.cs`, README có lệnh cài và chạy.

B. Cơ chế riêng của ASP.NET Core

- **Middleware pipeline và thứ tự**: mỗi middleware có thể xử lý trước và sau `next`; đảo thứ tự (ví dụ xác thực sau phân quyền) thì kết quả đổi ra sao. Nhóm chỉ được một tình huống thứ tự có ý nghĩa.
- **Dependency Injection tích hợp**: `AddSingleton`, `AddScoped`, `AddTransient`; hậu quả khi đăng ký sai lifetime (ví dụ service scoped bị singleton giữ).
- **Controller hay Minimal API**: chọn một cách, giải thích lý do chọn trong bối cảnh đồ án.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
EF Core: truy vấn, `Include`, `AsNoTracking`, migration	Có CSDL quan hệ	Migration chạy lại được; log SQL cho một truy vấn có `Include`
Authentication và authorization (policy, role, claim, JWT)	Có đăng nhập, nhiều vai trò	Hai vai trò truy cập cùng endpoint cho kết quả khác nhau; chỉ rõ quyết định phân quyền diễn ra ở đâu
Filter (action filter, exception filter)	Cần xử lý xuyên suốt ở tầng controller	Phân biệt rõ filter với middleware
Hosted/Background service	Có tác vụ nền	Log của tác vụ nền chạy; tắt ứng dụng thì dừng êm
Options pattern, cấu hình theo môi trường	Nhiều môi trường	Một cấu hình thay đổi giữa Development và Production
Health checks và logging	Cần vận hành	Endpoint health trả đúng; log có cấu trúc
Rate limiting, CORS	API công khai	Gửi nhiều request và nhận mã 429 đúng lúc

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Tối ưu truy vấn và cache	Tái hiện N+1 rồi sửa bằng `Include` hoặc projection; thêm `IMemoryCache` hoặc Redis; số truy vấn và thời gian phản hồi trước/sau.
Realtime với SignalR	Hub, quản lý kết nối, nhóm (group), xác thực kết nối; xử lý mất kết nối; nêu cách mở rộng nhiều server (backplane) và thử một ca.
Xử lý nền có hàng đợi	Channel hoặc hàng đợi ngoài; retry, trạng thái job, chống xử lý trùng.
Kiểm thử tích hợp	`WebApplicationFactory` với CSDL thử; test xác thực và phân quyền; chạy trong CI.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Quan sát hệ thống	OpenTelemetry hoặc log có cấu trúc kèm request id; truy lại một request lỗi xuyên các tầng.
Đóng gói và CI	Docker/docker compose chạy bằng một lệnh; GitHub Actions build và test.

4. Minh chứng cần có trong báo cáo và repo

- Thứ tự middleware của dự án và một tình huống mà thứ tự đó ảnh hưởng kết quả.
- Service được inject thật (không khởi tạo thủ công), kèm lifetime đã chọn và lý do.
- Nếu dùng EF Core, auth hoặc hosted service: migration, request mẫu và log minh chứng.
- Danh sách endpoint (Swagger/OpenAPI) và log độ trễ của các endpoint chính.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ dùng template mặc định rồi viết CRUD controller.
- Không phân biệt được middleware với filter.
- Đăng ký service sai lifetime nhưng không giải thích được hậu quả.

6. Câu hỏi tự kiểm

- Thứ tự các middleware trong pipeline của em có ý nghĩa gì? Đảo hai cái thì sao?
- Service này nên scoped hay singleton? Vì sao?
- Authorization của endpoint này được quyết định ở đâu?
- Điều gì xảy ra với `DbContext` sau mỗi request?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn ASP.NET Core

- **Spring Boot**: kiến trúc rất giống (DI, phân tầng); chọn ASP.NET Core nếu nhóm quen C#, và công cụ phát triển (Visual Studio, EF Core) gắn kết chặt.
- **NestJS, FastAPI, Express**: viết nhanh hơn, ít khai báo; ASP.NET Core mạnh ở kiểu dữ liệu chặt, hiệu năng cao và hệ sinh thái doanh nghiệp.
- **Go (Gin)**: nhẹ, khởi động nhanh; ASP.NET Core có nhiều thành phần dựng sẵn hơn (auth, ORM, DI).

Những trường hợp KHÔNG nên dùng ASP.NET Core (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Dịch vụ rất nhỏ chỉ vài endpoint: có thể dùng Minimal API nhưng so với FastAPI hay Express vẫn nhiều cấu hình hơn.
- Nhóm chưa quen C# và .NET: thời gian dành cho học ngôn ngữ và công cụ.
- Môi trường không hỗ trợ runtime .NET hoặc yêu cầu chạy cực nhẹ: cân nhắc Go hoặc Node.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Chèn một middleware mới và chứng minh vị trí đặt là đúng.
- Thêm một policy authorization cho endpoint có sẵn.
- Thêm migration và cập nhật API theo schema mới.

Spring Boot

Backend · Framework Java · Spring Boot 4.x (Spring Framework 7, Jakarta EE 11, Java 17 trở lên). Nhóm ghi rõ phiên bản đang dùng trong README.

> **Trọng tâm của đề tài**
> Điểm mạnh của Spring Boot là **IoC/Dependency Injection, tự cấu hình (auto-configuration) và hệ sinh thái Spring** (Data, Security, Test). Đề tài được đánh giá ở chỗ nhóm hiểu framework gọi code của mình khi nào và bean được tạo ra sao, không phải ở số bảng CRUD hay số endpoint.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Backend (trình bày theo thuật ngữ của Spring Boot)

- **Luồng một request**: client gọi vào, controller (`@RestController`, `@GetMapping`...) nhận, chuyển cho service, service gọi repository, kết quả trả về dạng JSON. Vẽ lại luồng này cho một chức năng thật của đồ án.
- **Phân tầng**: controller chỉ lo giao tiếp HTTP; service chứa nghiệp vụ; repository lo truy cập dữ liệu. Dùng **DTO** để tách dữ liệu trả về khỏi entity.
- **Truy cập dữ liệu**: Spring Data JPA, entity, quan hệ `@OneToMany`/`@ManyToOne`, repository, query method. Biết nhận ra vấn đề **N+1 query** và cách xử lý.
- **Lược đồ CSDL**: dùng migration (Flyway hoặc Liquibase); không chỉ dựa vào `ddl-auto` tự sinh bảng khi nộp đồ án.
- **Phản hồi lỗi có cấu trúc**: mã HTTP đúng (400, 401, 403, 404, 409...), định dạng lỗi thống nhất (ví dụ `ProblemDetail`).
- **Cấu hình**: `application.yml`, profile theo môi trường (dev, prod), bí mật (mật khẩu, khóa) đưa qua biến môi trường, có `.env.example`.
- **Kiểm thử đơn vị bằng JUnit 5**: viết test cho logic nghiệp vụ ở tầng service, có ca đúng và ca lỗi; chạy được bằng Maven hoặc Gradle.
- **Mockito để mock dependency** khi kiểm thử Service/Controller; phân biệt unit test (không khởi động Spring, mock phụ thuộc) với integration test (khởi động một phần hoặc toàn bộ ứng dụng).
- **Công cụ**: Maven hoặc Gradle, đóng gói file jar, README có lệnh cài và chạy.

B. Cơ chế riêng của Spring Boot

- **IoC và Dependency Injection**: bean được tạo bởi ai, lúc nào, phạm vi (scope) là gì; dùng constructor injection; nếu có hai cài đặt cùng một interface thì chọn bằng `@Primary` hay `@Qualifier`.
- **Auto-configuration và starter**: vì sao thêm `spring-boot-starter-data-jpa` là có sẵn `DataSource` và `EntityManager`; xem được điều kiện tự cấu hình (chạy với `--debug` hoặc dùng Actuator). Ở bản 4, framework được tách module nhỏ hơn nên một số tự cấu hình cần thêm starter riêng.
- **Vòng đời bean**: khởi tạo, `@PostConstruct`, hủy. Biết điều gì xảy ra nếu tự `new` một service thay vì để container quản lý.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Bean Validation (`@Valid`, `@NotBlank`, `@Size`...)	Hầu hết ứng dụng có nhận dữ liệu từ người dùng	Gửi dữ liệu sai và nhận phản hồi lỗi đúng định dạng, đúng mã HTTP
Spring Security (xác thực, phân quyền theo vai trò)	Có đăng nhập hoặc nhiều loại người dùng	Hai vai trò truy cập cùng endpoint cho kết quả khác nhau; token hoặc session hết hạn
`@Transactional`	Một thao tác cần ghi nhiều bảng nhất quán (đặt vé, tính giảm giá, đơn hàng)	Ca lỗi giữa chừng và dữ liệu được rollback; chỉ ra transaction bắt đầu và kết thúc ở đâu
`@RestControllerAdvice` (xử lý lỗi toàn cục)	Có nhiều controller	Một nơi xử lý lỗi thay vì try/catch rải rác
AOP	Ghi log hoặc đo thời gian xuyên suốt	Aspect chạy đúng điểm cắt; giải thích proxy hoạt động ra sao

Cơ chế	Nên chọn khi	Cần chứng minh
Integration test: `@WebMvcTest`/`MockMvc`, `@DataJpaTest`, `@SpringBootTest`, Testcontainers	Muốn kiểm thử tầng web hoặc tầng dữ liệu với Spring	Test controller và repository riêng lẻ; báo cáo kết quả chạy test; nêu rõ khác unit test ở đâu
Phân trang, sắp xếp, tìm kiếm với Spring Data	Danh sách dài	Endpoint trả theo trang; câu truy vấn thực sự được sinh ra (xem log SQL)
OpenAPI/Swagger (springdoc), Actuator	Cần tài liệu API và theo dõi sức khỏe dịch vụ	Trang Swagger gọi thử được; endpoint health trả đúng

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Kiểm soát đồng thời khi ghi	Dùng khóa lạc quan (`@Version`) hoặc khóa bi quan cho thao tác tranh chấp (giữ chỗ, mã giảm giá có số lượng giới hạn); viết test nhiều luồng chạy song song để chứng minh không bán vượt số lượng.
Tối ưu truy vấn	Tái hiện N+1 rồi sửa bằng `JOIN FETCH` hoặc `@EntityGraph`; báo cáo số câu SQL và thời gian trước/sau.
Cache	`@Cacheable` với Redis hoặc cache cục bộ; đo tỷ lệ trúng cache và thời gian phản hồi; nêu cách làm mới cache.
Quan sát hệ thống	Micrometer với Prometheus/Grafana hoặc log có cấu trúc; có một biểu đồ hoặc log chứng minh phát hiện được một sự cố giả lập.
Khả năng chịu lỗi	Thử lại và giới hạn đồng thời (`@Retryable`, `@ConcurrencyLimit` có sẵn trong Spring Framework 7) hoặc circuit breaker; mô phỏng dịch vụ ngoài lỗi.
Bảo mật nâng cao	OAuth2 Resource Server, method security (`@PreAuthorize`), kiểm thử phân quyền tự động.
Đóng gói và CI	Docker/docker compose chạy bằng một lệnh; GitHub Actions chạy build và test.

4. Minh chứng cần có trong báo cáo và repo

- Chỉ ra bean/service/repository nào do Spring quản lý và cách chúng được inject (đoạn code kèm giải thích).
- Có truy vấn JPA thật và migration tương ứng; log SQL cho ít nhất một chức năng chính.
- Kết quả chạy unit test (JUnit 5 + Mockito) cho ít nhất một service, có ca lỗi.
- Nếu dùng Security, Transaction, AOP hoặc Cache: có tình huống chạy chứng minh (kết quả thành công và kết quả lỗi).
- Danh sách endpoint (Swagger hoặc bảng mô tả), ảnh gọi thử và log độ trễ của các endpoint chính.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ tạo entity, repository và controller CRUD rồi để Spring tự lo, không giải thích được gì bên dưới.
- Tạo dependency bằng `new` thay vì để container quản lý.
- Dùng annotation nhưng không giải thích được lúc nào framework gọi và gọi cái gì.
- Dùng `ddl-auto=update` rồi không có lược đồ hay migration, không ai chạy lại được.
- Không có unit test nào cho tầng service, hoặc test nào cũng phải khởi động cả ứng dụng.

6. Câu hỏi tự kiểm

- Bean này do ai tạo? Nếu có hai cài đặt thì Spring inject cái nào, vì sao?
- Transaction bắt đầu và kết thúc ở đâu trong luồng này? Điều gì xảy ra nếu một lệnh ghi lỗi giữa chừng?
- Controller và service của em khác nhau về trách nhiệm ra sao? Vì sao không để nghiệp vụ trong controller?
- Hai người đặt cùng một chỗ vào cùng một thời điểm thì hệ thống của em xử lý thế nào?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Spring Boot

- **ASP.NET Core**: kiến trúc rất giống (DI, middleware, EF Core); chọn Spring Boot nếu nhóm quen Java, và vì hệ sinh thái tài liệu, thư viện doanh nghiệp rất lớn.
- **NestJS, FastAPI, Express**: viết nhanh hơn, ít khai báo hơn; chọn Spring Boot khi cần kiểu dữ liệu chặt, giao dịch phức tạp, hoặc cấu trúc phân tầng rõ ràng cho nhóm nhiều người.
- **Quarkus, Micronaut**: nhẹ và khởi động nhanh hơn; Spring Boot có cộng đồng và tài liệu nhiều hơn nên dễ tìm lời giải khi gặp lỗi.

Những trường hợp KHÔNG nên dùng Spring Boot (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Dịch vụ rất nhỏ vài endpoint hoặc script tự động: Spring Boot khởi động chậm, tốn bộ nhớ và nhiều khai báo so với FastAPI hoặc Express.
- Môi trường serverless cần khởi động tức thì: khởi động lạnh của JVM là điểm yếu (có thể giảm bằng GraalVM native nhưng phức tạp thêm).
- Nhóm chưa vững Java và lập trình hướng đối tượng: thời gian sẽ dành cho học ngôn ngữ thay vì học framework.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm validation và phản hồi lỗi toàn cục cho một endpoint.
- Thêm một vai trò hoặc chính sách truy cập trong Spring Security (nếu đồ án có xác thực).
- Viết một unit test có Mockito cho một service có sẵn; hoặc một test slice cho controller/repository; sửa một transaction đặt sai chỗ.

NestJS

Backend · Framework Node.js + TypeScript · NestJS 11; nhóm ghi rõ phiên bản và nền HTTP (Express hoặc Fastify).

> **Trọng tâm của đề tài**
> Điểm đặc trưng của NestJS là **kiến trúc module, dependency injection và chuỗi cơ chế xử lý request (Guard, Interceptor, Pipe, Filter)**. Một ứng dụng chỉ có controller và service CRUD thì chưa dùng NestJS đúng bản chất.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Backend (trình bày theo thuật ngữ của NestJS)

- **Luồng một request**: request vào controller, qua service, đến nguồn dữ liệu, rồi trả JSON với lỗi có cấu trúc. Vẽ lại cho một chức năng thật.
- **Routing và nhận dữ liệu**: decorator `@Controller`, `@Get`, `@Post`; `@Param`, `@Query`, `@Body`; mã HTTP đúng.
- **Validation đầu vào**: DTO kết hợp `class-validator` và `ValidationPipe` (hoặc thư viện tương đương).
- **Truy cập dữ liệu**: một lớp ORM/query (TypeORM, Prisma, Drizzle hoặc tương đương) với migration.
- **Cấu hình và công cụ**: `@nestjs/config`, biến môi trường, Nest CLI, TypeScript; README có lệnh cài và chạy.
- **Xử lý lỗi**: `HttpException` và lỗi nghiệp vụ thành phản hồi có cấu trúc.

B. Cơ chế riêng của NestJS

- **Module**: feature thuộc module nào; `imports`, `exports`, `providers`; module dùng chung.
- **Provider và Dependency Injection**: inject qua constructor, vì sao không tự `new` service; phạm vi (scope) mặc định là singleton, khi nào dùng request scope.
- **Vòng đời request**: thứ tự middleware, guard, interceptor (trước), pipe, handler, interceptor (sau), exception filter; nhóm vẽ được thứ tự này cho một endpoint thật.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
DTO + `ValidationPipe` (transform, whitelist)	Mọi endpoint nhận dữ liệu	Dữ liệu sai bị chặn trước khi vào nghiệp vụ; trường lạ bị loại
Guard (xác thực, phân quyền theo vai trò)	Có đăng nhập, nhiều vai trò	So sánh trước và sau khi gắn guard; không dùng `if/else` rải rác trong controller
Interceptor (log, chuẩn hóa response, cache, đo thời gian)	Cần xử lý xuyên suốt	Log hoặc response đã chuẩn hóa; giải thích khác Guard ở đâu
Exception Filter	Muốn xử lý lỗi tập trung	Một định dạng lỗi chung cho nhiều loại lỗi
Custom decorator	Lấy user/role/metadata gọn hơn	`@CurrentUser()` hoặc `@Roles()` được dùng thật
Dynamic module, config module	Cần cấu hình linh hoạt	Module cấu hình bằng `forRoot`/`forRootAsync`
ORM, quan hệ và transaction	Có CSDL	Ca giao dịch lỗi và dữ liệu được rollback

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Cache và hàng đợi	Cache bằng Redis (`cache-manager`) với TTL; hàng đợi BullMQ có retry và trạng thái job; số liệu trước/sau hoặc ca lỗi và thử lại.
WebSocket Gateway	Xác thực kết nối, phòng (room), xử lý ngắt kết nối; thử nhiều client.
So sánh nền HTTP	Đổi Express sang Fastify; đo throughput/latency bằng công cụ tải (k6, autocannon) và giải thích khác biệt.
Kiểm thử	Unit test bằng Jest cho provider, e2e test bằng Supertest cho endpoint; chạy trong CI.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Quan sát hệ thống	Log có cấu trúc kèm request id; truy lại một request lỗi.
Tài liệu API	Swagger (`@nestjs/swagger`) khớp DTO; ví dụ request/response.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ module hoặc mô tả request flow của một endpoint thật.
- Chỉ ra ít nhất một dependency được inject và scope của nó.
- Nếu dùng Guard, Pipe, Interceptor hoặc Filter: request mẫu chứng minh trước/sau khi cơ chế chạy.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ có controller và service CRUD, không giải thích được module và DI.
- Tự `new` service hay repository bên trong controller.
- Kiểm tra quyền bằng `if/else` lặp lại ở mọi controller nhưng vẫn khai đã học Guard.

6. Câu hỏi tự kiểm

- Guard và Interceptor khác nhau ở đâu, và vì sao em đặt kiểm tra quyền ở Guard?
- Provider này được tạo và inject ở scope nào?
- Input sai được chặn ở đâu trước khi vào business logic?
- Module A muốn dùng service của module B thì cần làm gì?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn NestJS

- **Express/Fastify thuần**: nhẹ, ít quy ước; NestJS đáng chọn khi cần cấu trúc, DI và code dễ bảo trì cho nhóm nhiều người.
- **Spring Boot, ASP.NET Core**: cùng tư tưởng DI và phân tầng; NestJS dùng TypeScript nên hợp nhóm đã quen hệ JavaScript.
- **FastAPI**: nhanh khi cần API kiểu dữ liệu chặt bằng Python.

Những trường hợp KHÔNG nên dùng NestJS (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Dịch vụ rất nhỏ vài endpoint: mã khai báo (module, decorator) nhiều hơn mức cần.
- Nhóm chưa quen TypeScript và decorator: thời gian học cú pháp lấn át công nghệ.
- Cần hiệu năng thô cực cao với ít lớp trung gian: Fastify thuần hoặc Go phù hợp hơn.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một Guard hoặc quy tắc vai trò vào module hiện có.
- Viết Pipe hoặc DTO validation cho trường dữ liệu mới.
- Thêm Interceptor đo thời gian hoặc chuẩn hóa response nếu đồ án đã dùng pattern này.

FastAPI

Backend · Framework Python · FastAPI với Pydantic v2; nhóm ghi rõ phiên bản Python và thư viện truy cập dữ liệu.

> **Trọng tâm của đề tài**
> Điểm đặc trưng của FastAPI là **API theo kiểu dữ liệu với Pydantic, hệ thống dependency (`Depends`) và I/O bất đồng bộ đúng chỗ**. Viết mọi endpoint là `async def` không có nghĩa là bất đồng bộ.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Backend (trình bày theo thuật ngữ của FastAPI)

- **Luồng một request**: request vào route, qua kiểm tra dữ liệu bằng Pydantic, qua dependency, đến logic và truy cập dữ liệu, rồi trả JSON.
- **Routing và nhận dữ liệu**: path, query, body; `APIRouter` để chia module; mã HTTP đúng.
- **Truy cập dữ liệu**: SQLAlchemy (hoặc ORM khác) với phiên làm việc (session) đúng loại (đồng bộ hoặc bất đồng bộ) và migration (Alembic).
- **Xử lý lỗi**: `HTTPException`, exception handler, định dạng lỗi thống nhất.
- **Cấu hình và công cụ**: `pydantic-settings` hoặc biến môi trường, `.env.example`; chạy bằng `uvicorn`; README.
- **Tài liệu API**: OpenAPI/Swagger do FastAPI tự sinh, đúng schema.

B. Cơ chế riêng của FastAPI

- **Pydantic**: model cho request và response; validation tự động; lỗi do dữ liệu sai kiểu sinh ra ở tầng nào và trả mã nào (422).
- **Dependency system (`Depends`)**: auth, phiên CSDL, cấu hình dùng chung; chỉ ra đồ thị dependency của ít nhất một endpoint.
- **`async` đúng chỗ**: endpoint I/O-bound nào dùng `await`; khi nào dùng `def` thường (FastAPI chạy trong thread pool); vì sao gọi code blocking (ví dụ SQLAlchemy đồng bộ, `time.sleep`) trong `async def` làm chặn cả event loop.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
`response_model`, lược đồ OpenAPI có chủ đích	Cần giấu trường nhạy cảm, tách model vào/ra	Response không lộ trường nội bộ; mô tả trên Swagger
Exception handler tập trung	Nhiều loại lỗi nghiệp vụ	Một định dạng lỗi chung
`BackgroundTasks`	Việc ngắn sau khi trả response (gửi email)	Phân biệt với hàng đợi thật; giải thích giới hạn
Middleware (log, CORS, đo thời gian)	Cần xử lý xuyên suốt	Log độ trễ mỗi request
`lifespan` (khởi tạo/giải phóng tài nguyên)	Có kết nối CSDL, nạp mô hình	Tài nguyên được tạo một lần khi khởi động
`TestClient`/`httpx`, `dependency_overrides`	Luôn nên có	Test endpoint với dependency giả
Xác thực (OAuth2 + JWT)	Có đăng nhập	Ca token hợp lệ, hết hạn, sai quyền

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Đo ảnh hưởng của async	Dùng công cụ tải (k6, locust, wrk) so sánh một endpoint đúng/sai kiểu (blocking trong `async def`) với bản sửa; báo cáo throughput và độ trễ p95.
Cache và tối ưu truy vấn	Redis hoặc cache cục bộ; số truy vấn và độ trễ trước/sau; xử lý N+1.
Hàng đợi xử lý nền	Celery, RQ hoặc ARQ; retry, trạng thái job, chống chạy trùng.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Streaming và WebSocket	`StreamingResponse` hoặc SSE, WebSocket có xác thực; xử lý ngắt kết nối.
Quan sát hệ thống	Log có cấu trúc kèm request id; metrics hoặc tracing; truy lại một request lỗi.
Kiểm thử và CI	pytest + httpx; GitHub Actions; báo cáo độ phủ cho phần nghiệp vụ.

4. Minh chứng cần có trong báo cáo và repo

- Trang OpenAPI/Swagger chạy được và đúng schema.
- Đồ thị dependency của ít nhất một endpoint.
- Một endpoint `async` và giải thích `await` đang chờ I/O gì; một endpoint đồng bộ và lý do.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Viết mọi endpoint `async def` nhưng bên trong toàn code blocking.
- Không dùng Pydantic và `Depends` mà xử lý `dict` và input thủ công.
- Dùng `BackgroundTasks` cho việc dài hoặc hàng đợi nặng mà không hiểu giới hạn.

6. Câu hỏi tự kiểm

- Endpoint nào của em nên là async và endpoint nào không? Vì sao?
- Dependency này được tạo ở đâu và dùng lại thế nào?
- Nếu model input sai kiểu, lỗi được sinh ra ở tầng nào?
- Nếu em gọi SQLAlchemy đồng bộ trong `async def` thì chuyện gì xảy ra với các request khác?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn FastAPI

- **Flask, Django REST Framework**: Flask nhẹ hơn nhưng không có sẵn validation và tài liệu tự sinh; DRF đầy đủ hơn nhưng nặng và theo mô hình Django.
- **NestJS, Spring Boot, ASP.NET Core**: kiểu dữ liệu chặt, DI mạnh; FastAPI ngắn gọn hơn và hợp nhóm quen Python hoặc cần ghép với thư viện AI.
- **Express**: JavaScript, ít quy ước; FastAPI có validation và tài liệu API sẵn.

Những trường hợp KHÔNG nên dùng FastAPI (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Ứng dụng web có giao diện render phía server cùng admin, ORM, auth, form đầy đủ: Django phù hợp hơn.
- Công việc chủ yếu là tính toán CPU nặng trong tiến trình API: async không giúp; cần tiến trình hoặc hàng đợi riêng.
- Nhóm chưa hiểu kiểu dữ liệu (type hints) trong Python: Pydantic khó tận dụng.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm dependency auth hoặc config vào một endpoint hiện có.
- Thêm response model và validation cho một luồng thật.
- Chuyển một đoạn blocking không phù hợp sang cách xử lý đúng hơn.

Express.js

Backend · Web framework Node.js · Express 5; nhóm ghi rõ phiên bản Node.js và có dùng TypeScript hay không.

> **Trọng tâm của đề tài**
> Express rất nhẹ và không ép kiến trúc, nên chiều sâu phải đến từ việc **nhóm tự thiết kế middleware, luồng xử lý lỗi, validation, truy cập dữ liệu và các năng lực server mà ứng dụng thật cần**. Đồ án chỉ dừng ở CRUD cơ bản thì khối lượng công nghệ không đủ, nên phần Tự chọn bên dưới nên được chọn đầy đủ và làm kỹ.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Backend (trình bày theo thuật ngữ của Express)

- **Luồng một request**: request đi qua chuỗi middleware, đến route handler, đến service/tầng dữ liệu, rồi trả JSON. Vẽ luồng này cho một chức năng thật.
- **Routing**: `express.Router`, route params, query, body (`express.json()`); mã HTTP đúng.
- **Validation đầu vào**: kiểm tra dữ liệu bằng schema (Zod, Joi, express-validator…); lỗi trả mã 400 đúng định dạng.
- **Truy cập dữ liệu**: một lớp truy cập (ORM hoặc query builder) có migration/schema; không chỉ lưu trong bộ nhớ.
- **Cấu hình và công cụ**: biến môi trường, `.env.example`, `npm` scripts, README có lệnh cài và chạy.
- **Cấu trúc dự án**: nhóm chọn một cấu trúc (router/controller/service/repository) và giải thích vì sao, vì Express không ép kiến trúc.

B. Cơ chế riêng của Express

- **Middleware pipeline và thứ tự**: mỗi middleware có `(req, res, next)`; middleware gọi `next()` hoặc kết thúc response; đảo thứ tự thì kết quả đổi (ví dụ đặt xác thực sau route).
- **Xử lý lỗi bất đồng bộ tập trung**: middleware lỗi có 4 tham số `(err, req, res, next)`; lỗi trong hàm async đi qua đó thế nào (Express 5 tự chuyển lỗi của promise bị reject); tránh `unhandledRejection` và `try/catch` lặp ở mọi route.
- **Phân tách trách nhiệm**: route handler mỏng, nghiệp vụ nằm ở service; không viết handler dài hàng trăm dòng.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Do Express có lõi framework tương đối nhỏ, nhóm cần chọn đủ các cơ chế ở phần này để chứng minh một backend hoàn chỉnh (validation, bảo mật, xác thực, tầng dữ liệu, kiểm thử…); giảng viên đánh giá theo độ sâu thực tế. Phần Nâng cao vẫn là tự chọn.

Cơ chế	Nên chọn khi	Cần chứng minh
Validation bằng schema (Zod/Joi)	Mọi endpoint nhận dữ liệu	Dữ liệu sai bị chặn; thông báo lỗi chuẩn hóa
`helmet`, rate limit, CORS theo mối đe dọa thật	API công khai	Giải thích mối đe dọa nào mỗi cấu hình chặn; ca vượt giới hạn trả 429
Xác thực và phân quyền (JWT hoặc session)	Có đăng nhập	Hai vai trò, ca token hết hạn; lưu mật khẩu băm đúng cách
Tầng dữ liệu và migration	Có CSDL	Migration chạy lại được; transaction cho thao tác nhiều bước
Kiểm thử API và mock dependency	Luôn nên có	Test bằng Supertest; mock tầng dữ liệu; kết quả chạy
Upload hoặc streaming file	Có tệp lớn	Dùng stream; chứng minh không đọc cả tệp vào RAM

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Cache và hàng đợi	Redis cache có TTL và làm mới; hàng đợi (BullMQ) cho tác vụ chậm với retry, trạng thái job, chống xử lý trùng; số liệu trước/sau.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Realtime	Socket.IO hoặc WebSocket có xác thực, phòng (room), xử lý mất kết nối; dùng Redis adapter khi chạy nhiều tiến trình; thử nhiều client. Với gọi thoại WebRTC: máy chủ chỉ làm signaling (chuyển SDP/ICE), cần STUN/TURN; nêu rõ phần nào server làm, phần nào peer-to-peer.
Kiểm thử và chất lượng	Unit và integration test, độ phủ cho phần nghiệp vụ; chạy trong CI; ESLint.
Tìm kiếm hoặc tích hợp bên thứ ba	Full-text search hoặc webhook có xác thực chữ ký, retry, chống xử lý lặp.
Quan sát hệ thống	Log có cấu trúc (pino) kèm request id; health check; truy lại một request lỗi.
Hiệu năng	Công cụ tải (autocannon, k6); chỉ ra điểm nghẽn (event loop bị chặn do tác vụ CPU) và cách xử lý (worker thread, tách dịch vụ); số liệu trước/sau.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ request qua các middleware quan trọng.
- Một ví dụ lỗi async đi qua error handler (ảnh hoặc log).
- Nếu dùng auth, bảo mật hoặc tầng dữ liệu: test hoặc request chứng minh.
- Danh sách endpoint và log độ trễ của endpoint chính.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ CRUD lưu trong bộ nhớ với vài route, không có validation, xử lý lỗi hay bảo mật.
- Dùng middleware nhưng không hiểu thứ tự.
- Đưa toàn bộ logic vào route handler dài hàng trăm dòng.

6. Câu hỏi tự kiểm

- Request này đi qua những middleware nào theo thứ tự?
- Nếu service ném lỗi bất đồng bộ thì ai bắt?
- Express không ép kiến trúc; tại sao nhóm chọn cấu trúc thư mục này?
- Phần nào của hệ thống của em khiến Express "không đủ" và em đã bù bằng gì?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Express

- **NestJS**: cùng nền Node.js nhưng có module, DI và cấu trúc sẵn; chọn Express khi muốn nhẹ và tự thiết kế, chấp nhận phải bù bằng kỷ luật của nhóm.
- **Fastify, Hono**: nhanh và có schema; Express có hệ sinh thái middleware lớn nhất.
- **FastAPI, Spring Boot, ASP.NET Core**: có validation và DI dựng sẵn; Express đổi lại ít khái niệm khi bắt đầu.

Những trường hợp KHÔNG nên dùng Express (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Hệ thống lớn nhiều người cùng làm cần kiến trúc chuẩn ép buộc: NestJS hoặc Spring Boot phù hợp hơn.
- Tác vụ nặng về CPU trong tiến trình chính: event loop đơn luồng bị chặn, cần worker hoặc dịch vụ riêng.
- Chỉ cần vài endpoint tĩnh hoặc proxy đơn giản: thậm chí không cần framework.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm middleware auth, rate-limit hoặc logging vào luồng có sẵn.
- Thêm validation schema cho endpoint và chuẩn hóa lỗi.
- Chuyển một route handler quá lớn thành các lớp hoặc module hợp lý.

Django

Backend · Framework Python có sẵn nhiều thành phần (ORM, Admin, auth) · Django 5.2 LTS hoặc 6.x; có hoặc không kèm Django REST Framework (DRF). Nhóm ghi rõ phiên bản.

> **Trọng tâm của đề tài**
> Điểm mạnh của Django là bộ sẵn có: **ORM, migration, Admin, auth, form, template**. Đề tài được đánh giá ở chỗ nhóm dùng đúng bản chất các thành phần đó thay vì viết lại từ đầu; không dùng Django chỉ làm bộ định tuyến rồi tự viết SQL và CRUD.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Backend (trình bày theo thuật ngữ của Django)

- **Luồng một request**: URL conf, view, model (ORM), template hoặc serializer (DRF), response. Vẽ lại cho một chức năng thật.
- **Routing**: `urls.py`, `path()`, tham số URL; chia ứng dụng theo app.
- **Truy cập dữ liệu**: model, quan hệ (`ForeignKey`, `ManyToMany`), migration, QuerySet (lọc, sắp xếp, tổng hợp); QuerySet là lười (lazy) nghĩa là gì.
- **Xử lý lỗi và biểu mẫu**: validation của model/form hoặc serializer; trả mã và thông báo đúng.
- **Cấu hình**: `settings.py` tách theo môi trường, bí mật qua biến môi trường, `.env.example`.
- **Công cụ**: `manage.py`, môi trường ảo, `requirements.txt` hoặc tương đương, README.

B. Cơ chế riêng của Django

- **ORM và migration**: vì sao dùng ORM thay vì ghép SQL; migration sinh ra và áp dụng thế nào; `select_related` và `prefetch_related` để tránh N+1.
- **Django Admin đúng bản chất**: dùng để quản trị dữ liệu nội bộ (đăng ký model, tùy biến `ModelAdmin`), khác giao diện người dùng cuối; không tự viết lại trang quản trị CRUD chỉ để tăng số trang.
- **Một hướng giao diện rõ ràng**: Django Templates (server-rendered) hoặc DRF (API); giải thích vì sao.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
DRF: serializer, viewset, router	Xây API	Một viewset đầy đủ; phân trang; định dạng lỗi
Django Forms/ModelForm	Giao diện render phía server có biểu mẫu	Form có validation và hiển thị lỗi
Auth và permission (`django.contrib.auth`, DRF permissions)	Có đăng nhập, nhiều vai trò	Hai vai trò truy cập cùng chức năng cho kết quả khác nhau
Middleware tùy chỉnh	Xử lý xuyên suốt	Middleware chạy đúng vị trí, giải thích thứ tự
Signals	Cần phản ứng theo sự kiện của model	Chỉ rõ trigger và ca lỗi; nêu lý do không gọi trực tiếp trong view/service
Cache tích hợp, Redis	Dữ liệu đọc nhiều	Số truy vấn và thời gian trước/sau
Celery hoặc RQ cho tác vụ nền	Việc chậm (gửi email, xử lý tệp)	Job chạy, thử lại khi lỗi, xem được trạng thái

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Tối ưu truy vấn	Dùng django-debug-toolbar hoặc `assertNumQueries`; tái hiện N+1, sửa bằng `select_related`/`prefetch_related`; số truy vấn và thời gian trước/sau; thêm index và đo.
Cache nhiều tầng	Cache theo view hoặc theo mức thấp bằng Redis, quy tắc làm mới; tỷ lệ trúng cache.
Realtime với Channels	WebSocket có xác thực, nhóm (group), xử lý ngắt kết nối.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Bảo mật	Chạy `manage.py check --deploy` và sửa cảnh báo; CSRF, XSS, quyền; test các ca bị chặn.
Kiểm thử	Test model, view, API; chạy trong CI; báo cáo độ phủ cho phần nghiệp vụ.
Triển khai	Docker/docker compose với CSDL thật, tệp tĩnh, biến môi trường; chạy lại bằng một lệnh.

4. Minh chứng cần có trong báo cáo và repo

- Migration và truy vấn QuerySet thật (kèm log SQL cho chức năng chính).
- Admin hoặc DRF chạy được và đúng vai trò.
- Nếu dùng signal, cache hoặc hàng đợi: giải thích trigger và ca lỗi.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ dùng Django như một bộ định tuyến rồi viết SQL và CRUD thủ công.
- Tự viết lại hệ thống auth hoặc admin mà không có lý do học thuật hay kỹ thuật.
- Dùng signal cho mọi thứ khiến luồng khó truy vết.

6. Câu hỏi tự kiểm

- Tại sao dùng QuerySet/ORM thay vì tự ghép SQL ở đây?
- Admin của Django dùng cho ai và khác giao diện người dùng cuối thế nào?
- Signal này có thật sự cần hay có thể gọi trực tiếp trong service hoặc view?
- QuerySet này chạy bao nhiêu truy vấn? Em đo bằng cách nào?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Django

- **FastAPI, Flask**: nhẹ hơn và hợp API thuần; Django hợp khi cần sẵn admin, auth, ORM, form và ứng dụng web đầy đủ.
- **Laravel, Spring Boot, ASP.NET Core**: cùng kiểu framework đầy đủ ở ngôn ngữ khác; Django đáng chọn khi nhóm quen Python.
- **Ruby on Rails**: tương tự về triết lý có sẵn nhiều thành phần, nhưng ít phổ biến trong cộng đồng sinh viên hiện nay.

Những trường hợp KHÔNG nên dùng Django (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- API thuần hiệu năng cao, nhiều I/O bất đồng bộ: FastAPI phù hợp hơn.
- Dịch vụ nhỏ chỉ vài endpoint: Django mang theo nhiều thứ không dùng đến.
- Giao diện frontend tách riêng hoàn toàn và không cần admin: framework nhẹ hơn là đủ.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm migration và truy vấn lọc phù hợp.
- Thêm permission cho một endpoint DRF hoặc một luồng form.
- Tối ưu một truy vấn N+1 bằng `select_related`/`prefetch_related` nếu đồ án có.

Go + Gin

Backend · Ngôn ngữ Go kèm web framework Gin (hoặc Echo, Fiber) · Go phiên bản ổn định mới; nhóm ghi rõ phiên bản Go và Gin.

> **Trọng tâm của đề tài**
> Chiều sâu của Go nằm ở **`context.Context`, middleware, xử lý lỗi theo phong cách Go, interface và (khi bài toán thật sự cần) xử lý đồng thời bằng goroutine và channel**. Nếu bài toán không có nhu cầu đồng thời thật thì nhóm nói rõ điều đó trong báo cáo, không nhét goroutine cho có.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng Backend (trình bày theo thuật ngữ của Go và Gin)

- **Luồng một request**: request vào router Gin, qua middleware, vào handler, vào tầng nghiệp vụ, vào tầng dữ liệu, rồi trả JSON.
- **Routing và nhận dữ liệu**: nhóm route (`router.Group`), tham số đường dẫn, query, `ShouldBindJSON`; validation bằng struct tag.
- **Xử lý lỗi theo phong cách Go**: hàm trả `error`; bọc lỗi bằng `%w`; kiểm tra bằng `errors.Is`/`errors.As`; chuyển lỗi thành mã HTTP ở một nơi tập trung.
- **Truy cập dữ liệu**: `database/sql` kèm `sqlc`, hoặc GORM; migration; đóng kết nối đúng cách.
- **Cấu hình và công cụ**: biến môi trường, `go mod`, `gofmt`, `go vet`, `go test`; README có lệnh cài và chạy.

B. Cơ chế riêng của Go

- **`context.Context`**: truyền qua các tầng, dùng để hủy và đặt thời hạn (timeout) cho truy vấn CSDL và gọi dịch vụ ngoài; nếu bỏ `context` đi thì hỏng chỗ nào.
- **Middleware của Gin**: thứ tự chuỗi middleware, `c.Next()` và `c.Abort()`; viết ít nhất một middleware tự viết (log, xác thực hoặc đo thời gian).
- **Interface và cấu trúc dự án**: tầng nghiệp vụ phụ thuộc vào interface, không phụ thuộc trực tiếp vào cài đặt; tiêm phụ thuộc (dependency injection) thủ công qua constructor; `struct` embedding ở mức dùng được.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Goroutine và channel chỉ có giá trị khi bài toán cần: ví dụ nhận trạng thái từ nhiều thiết bị cùng lúc, xử lý tác vụ nền theo hàng đợi, gọi nhiều dịch vụ song song. Không có nhu cầu như vậy thì chuyển chiều sâu sang các mục khác.

Cơ chế	Nên chọn khi	Cần chứng minh
Goroutine, channel, `sync.WaitGroup`, `sync.Mutex`	Bài toán có xử lý đồng thời thật	Mô tả luồng đồng thời; chạy `go test -race` không báo lỗi; giải thích vì sao cần
Worker pool, hàng đợi trong tiến trình	Tác vụ nền cần giới hạn số worker	Số worker giới hạn; tác vụ không mất khi tắt êm
Graceful shutdown	Luôn nên có với dịch vụ	Gửi tín hiệu tắt; request đang chạy hoàn tất; kết nối được đóng
Table-driven test	Luôn nên có	Test bảng cho một hàm có nhiều ca; ca lỗi
Xác thực (JWT middleware) và phân quyền	Có đăng nhập	Hai vai trò; ca token sai hoặc hết hạn
Struct embedding, kiểu tùy chỉnh	Muốn tái sử dụng hành vi	Dùng có mục đích, không lạm dụng
Cấu hình theo môi trường	Nhiều môi trường	Cấu hình đọc từ biến môi trường, có giá trị mặc định hợp lý

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Đo hiệu năng và profiling	Dùng công cụ tải (k6, hey, wrk) để đo throughput và độ trễ p50/p95; dùng `pprof` để tìm hotspot CPU hoặc bộ nhớ; số liệu trước/sau một tối ưu.
Đồng thời có kiểm soát	Giới hạn đồng thời, backpressure (kênh có đệm, bỏ bớt tải); kiểm tra race, rò rỉ goroutine.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Cache và dữ liệu	Redis hoặc cache trong bộ nhớ có TTL; truy vấn và index tối ưu; số liệu trước/sau.
Quan sát hệ thống	`log/slog` có cấu trúc kèm request id, metrics (Prometheus), health check.
Đóng gói và CI	Dockerfile nhiều giai đoạn (multi-stage); docker compose chạy bằng một lệnh; GitHub Actions chạy build, vet và test.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ request qua router, middleware, handler, tầng nghiệp vụ, tầng dữ liệu.
- Chỉ ra `context.Context` được truyền và dùng ở đâu (ví dụ truy vấn CSDL có thời hạn).
- Nếu dùng goroutine và channel: sơ đồ luồng đồng thời và kết quả `go test -race ./...`.
- Danh sách endpoint (ví dụ OpenAPI hoặc bảng mô tả) và log độ trễ của endpoint chính.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ dùng Gin viết CRUD, không giải thích được `context`, middleware hay xử lý lỗi.
- Dùng `panic` thay cho trả `error`, hoặc bỏ qua lỗi bằng `_`.
- Khai goroutine và channel nhưng bài toán không có nhu cầu đồng thời, hoặc chạy `-race` báo lỗi mà không biết.

6. Câu hỏi tự kiểm

- `context.Context` trong handler của em dùng để làm gì? Bỏ đi thì hỏng chỗ nào?
- Goroutine này kết thúc khi nào? Nếu client hủy request thì nó có bị dọn không?
- Hai goroutine cùng ghi vào biến này thì chuyện gì xảy ra? Em bảo vệ bằng cách nào?
- Vì sao tầng nghiệp vụ phụ thuộc vào interface thay vì kiểu cụ thể?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Go và Gin

- **Echo, Fiber, `net/http` chuẩn**: cùng phong cách; Gin phổ biến và nhiều tài liệu nhất. Với `net/http` hiện đại, nhóm cần nêu Gin đem lại gì (binding, middleware tiện).
- **Spring Boot, ASP.NET Core, NestJS**: nhiều thành phần dựng sẵn hơn; Go đổi lại nhẹ, khởi động nhanh, xử lý đồng thời tự nhiên.
- **Node.js (Express/Fastify)**: viết nhanh hơn; Go mạnh ở tải đồng thời và triển khai một tệp nhị phân.

Những trường hợp KHÔNG nên dùng Go và Gin (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Ứng dụng CRUD thuần, không có nhu cầu hiệu năng hay đồng thời: Go không thể hiện được thế mạnh, framework nhiều thành phần dựng sẵn viết nhanh hơn.
- Cần ORM mạnh, migration và admin sẵn có: Django hoặc Laravel tiện hơn.
- Nhóm chưa quen kiểu dữ liệu tĩnh và cách xử lý lỗi tường minh: thời gian dành cho học ngôn ngữ.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một middleware (xác thực, log, đo thời gian) vào luồng có sẵn.
- Thêm `context` có timeout vào một truy vấn đang không có.
- Sửa một lỗi race hoặc rò rỉ goroutine trong mã hiện có.

Go + gRPC

Backend & API · gRPC với Go (Protocol Buffers) · Go phiên bản ổn định mới; nhóm ghi rõ phiên bản Go, `protoc`/`buf` và thư viện gRPC.

> **Trọng tâm của đề tài**
> Trọng tâm là **gRPC và xử lý đồng thời trong Go**. Trò chơi (ví dụ cờ caro, hay XO 3×3, Connect 4…) chỉ là bối cảnh minh họa, **không phải trọng tâm chấm điểm**. Không bắt buộc giao diện đồ họa đầy đủ, AI đánh cờ, luật nâng cao hay xếp hạng phức tạp; client dòng lệnh hoặc giao diện tối giản là đủ. Nhóm chốt kiến trúc client ngay từ đầu: nếu client là web, cần gRPC-Web hoặc một gateway/proxy tương thích; nếu client là desktop hoặc dòng lệnh thì có thể dùng gRPC thuần.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. gRPC

- **Thiết kế `.proto` rõ ràng**: message, service, quy ước đặt tên; hiểu vì sao dữ liệu nhị phân Protocol Buffers khác JSON.
- **Unary RPC** cho thao tác một lần (tạo phòng, vào phòng, xem trạng thái) và **streaming RPC** cho luồng cập nhật liên tục. Gameplay ưu tiên **bidirectional streaming**.
- **Status code, deadline và cancellation**: trả mã gRPC đúng cho từng lỗi; client đặt deadline; khi mất kết nối hoặc stream kết thúc, vòng đời stream được xử lý đúng.

B. Go (đồng thời và quản lý tài nguyên)

- **Goroutine và đồng bộ trạng thái dùng chung** bằng channel hoặc `sync.Mutex`/`RWMutex`, chọn cái nào và vì sao.
- **`context.Context`**: dùng cho hủy và hết hạn, dọn tài nguyên; kiểm tra race bằng `go test -race ./...`.
- **Xử lý lỗi theo phong cách Go** (trả `error`, bọc `%w`, `errors.Is/As`) và dùng interface để tách tầng nghiệp vụ khỏi tầng truyền thông.

C. Trạng thái trò chơi tối thiểu (làm bối cảnh)

- **Server giữ trạng thái chuẩn (authoritative)**: kiểm tra đúng lượt, ô hợp lệ, điều kiện thắng; client không được tự quyết kết quả.
- **Phát hiện mất kết nối và giải phóng tài nguyên** đúng; khả năng kết nối lại và tiếp tục ván không bắt buộc.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Interceptor (xác thực, log, giới hạn tần suất)	Muốn xử lý xuyên suốt	Mọi RPC đi qua interceptor; ca token sai
gRPC reflection	Muốn thử không cần mã client	Gọi RPC bằng `grpcurl` ở chế độ reflection
Kết nối lại và tiếp tục ván	Mạng không ổn định	Ngắt kết nối giả lập, vào lại và tiếp tục đúng trạng thái
Lưu lịch sử hoặc trạng thái ván	Cần xem lại ván	Lưu và đọc lại; nhất quán sau khi khởi động lại
Người xem bằng server-streaming	Có chế độ xem ván	Hai người xem nhận cùng chuỗi nước đi
Hàng đợi ghép trận (matchmaking)	Nhiều người chờ	Hai người vào hàng đợi được ghép; xử lý khi người chờ thoát
gRPC-Web hoặc gateway/proxy tương thích	Client là web	Giải thích proxy làm gì; client web gọi được server

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Đo độ trễ và tải	Đo độ trễ p50/p95 của nước đi; kiểm thử tải với nhiều stream đồng thời (ví dụ ghz hoặc client tự viết); chỉ ra điểm nghẽn.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Bảo mật truyền thông	TLS hoặc mTLS; cấu hình chứng chỉ; chứng minh kết nối không mã hóa bị từ chối.
Tắt êm và áp lực ngược (backpressure)	Graceful shutdown cho stream đang chạy; xử lý client đọc chậm làm đầy hàng đợi.
Nhiều instance và trạng thái dùng chung	Chạy nhiều server với Redis hoặc kho dùng chung; xử lý ván nằm ở instance nào.
Quan sát hệ thống	Log có cấu trúc, metrics, tracing; truy lại một ván lỗi.
Phiên bản hóa protobuf và xếp hạng	Đổi `.proto` tương thích ngược (thêm trường, đánh dấu loại bỏ); hoặc xếp hạng ELO/MMR đơn giản có kiểm thử.

4. Minh chứng cần có trong báo cáo và repo

- Tệp `.proto` và danh sách RPC (unary, streaming) kèm mô tả mục đích.
- Ảnh hoặc log gọi RPC bằng `grpcurl` hoặc client thử, kể cả ca lỗi (sai lượt, ô đã có quân, mất kết nối).
- Kết quả `go test -race ./...` và ít nhất một integration test mô phỏng hai client chơi qua gRPC.
- Log độ trễ của nước đi; docker compose chạy được bằng một lệnh; GitHub Actions chạy build và test; README mô tả RPC và cách chạy client thử.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể. Dòng Tầng 1 không được ghi "mini-demo thay thế".

5. Chưa được xem là đạt nếu chỉ…

- Chỉ có unary RPC, không có streaming, nhưng khai đã làm "thời gian thực".
- Client quyết định kết quả ván hoặc server không kiểm tra lượt và điều kiện thắng.
- Không xử lý ngắt kết nối nên goroutine và stream bị rò rỉ; chạy `-race` báo lỗi mà không biết.
- Dồn thời gian vào giao diện hoặc luật trò chơi thay vì gRPC và đồng thời.

6. Câu hỏi tự kiểm

- Nếu thay gRPC bằng REST thì phần nào của hệ thống em hỏng hoặc phải làm khác?
- Khi client mất kết nối giữa chừng, `context` của stream đóng ở đâu và goroutine nào được dọn?
- Hai nước đi đến cùng lúc thì server xử lý thế nào? Em bảo vệ trạng thái ván bằng gì?
- Deadline và cancellation khác nhau thế nào? Em dùng ở đâu?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn gRPC

- **REST và WebSocket**: REST hợp thao tác đơn lẻ; WebSocket hợp trình duyệt và hai chiều nhưng không có hợp đồng kiểu dữ liệu; gRPC cho hợp đồng chặt, streaming hai chiều và nhiều ngôn ngữ.
- **Socket thuần hoặc thư viện như Socket.IO**: linh hoạt, nhưng phải tự thiết kế giao thức, mã hóa dữ liệu và xử lý lỗi.
- **Kafka, NATS, MQTT**: hướng thông điệp nhiều bên; gRPC hợp lời gọi có phản hồi và luồng giữa hai đầu.

Những trường hợp KHÔNG nên dùng gRPC (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Client chủ yếu là trình duyệt và cần đơn giản: gRPC-Web thêm một lớp proxy, WebSocket hoặc REST dễ hơn.
- API công khai cho bên thứ ba dùng bằng công cụ phổ thông (curl, Postman): REST dễ khám phá hơn.
- Bài toán không có luồng hai chiều hoặc nhu cầu hiệu năng: lợi ích của gRPC không thể hiện ra.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một RPC unary mới có kiểm tra lỗi và mã trạng thái đúng.
- Giải thích rồi sửa một chỗ stream không được dọn khi client ngắt kết nối.
- Viết một test (hoặc chạy `-race`) cho một đoạn có trạng thái dùng chung.

LlamaIndex (hồ sơ RAG)

Ứng dụng AI · Thư viện lập chỉ mục và truy hồi dữ liệu · Python · ghi rõ phiên bản `llama-index-core` trong requirements

> **Trọng tâm của đề tài**
> RAG là kiến trúc ứng dụng, **LlamaIndex là công nghệ cần học**; bộ tài liệu chỉ là dữ liệu minh họa. Đề tài hợp lý là hỏi đáp hoặc tra cứu tài liệu, không phải một hệ chuyên gia khó về nghiệp vụ. Điểm được chấm ở **việc nhìn thấy và đo được từng khâu của pipeline**, không phải ở việc câu trả lời có vẻ đúng lúc demo.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của ứng dụng AI

- **Pipeline thật của đồ án**: vẽ sơ đồ nạp dữ liệu, cắt đoạn, embedding và lập chỉ mục, truy hồi, tạo ngữ cảnh, sinh câu trả lời, trích nguồn, đánh giá.
- **Cấu hình mô hình**: dùng LLM nào và embedding nào, chạy bằng API hay chạy cục bộ; tham số quan trọng (nhiệt độ, độ dài ngữ cảnh); chi phí và độ trễ thực tế. Với tài liệu tiếng Việt, nêu lý do chọn embedding đa ngôn ngữ.
- **Dữ liệu vào/ra và giới hạn**: nguồn dữ liệu, cách làm sạch, những loại câu hỏi hệ thống trả lời được và không trả lời được.

B. Cơ chế riêng của LlamaIndex

- **Nạp dữ liệu**: `Document`, reader (ví dụ `SimpleDirectoryReader`), metadata (tên file, trang, chương) gắn ngay từ lúc nạp.
- **Chia thành Node**: `SentenceSplitter` hoặc node parser khác; giải thích `chunk_size` và `chunk_overlap` ở mức ứng dụng; in ra nội dung vài node để xem tận mắt.
- **Embedding và vector index**: `VectorStoreIndex`; vector lưu biểu diễn gì và tra cứu bằng độ tương đồng nào.
- **Retriever**: từ câu hỏi lấy ra các node liên quan; hiểu `similarity_top_k`; in ra node và điểm tương đồng, không chỉ in câu trả lời cuối.
- **Query engine và tổng hợp câu trả lời**: dùng ngữ cảnh đã truy hồi để sinh câu trả lời; mẫu prompt; truy ngược về nguồn qua `source_nodes`.
- **Từ chối có căn cứ**: khi câu hỏi nằm ngoài tài liệu, hệ thống nói không tìm thấy thay vì bịa.
- **Đánh giá thực nghiệm tối thiểu**: một bộ câu hỏi kiểm thử (gợi ý từ 20 câu, có cả câu ngoài tài liệu) và bảng kết quả truy hồi, hoặc độ bám nguồn của câu trả lời. LlamaIndex có sẵn công cụ đánh giá (ví dụ evaluator về faithfulness, relevancy, và đo hit rate/MRR cho retrieval).

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Lọc theo metadata	Có nhiều loại tài liệu, năm, chương hoặc đối tượng	Cùng một câu hỏi, kết quả khác nhau khi lọc và không lọc
Hiển thị trích dẫn và mở đoạn nguồn	Người dùng cần kiểm chứng câu trả lời	Mỗi câu trả lời có nguồn bấm mở được
Lưu index (persist)	Dữ liệu nhiều, không muốn embedding lại mỗi lần chạy	Thời gian khởi động trước và sau; tải lại index từ đĩa
Node parser riêng theo cấu trúc tài liệu (markdown, tiêu đề, bảng)	Tài liệu có cấu trúc rõ, cắt thô làm mất mạch	So sánh kết quả truy hồi giữa cắt thô và cắt theo cấu trúc
Nhiều nguồn hoặc nhiều index, `RouterQueryEngine`	Dữ liệu thuộc loại khác nhau (bảng và văn bản)	Log cho thấy câu hỏi được chuyển đúng nhánh
Hội thoại và viết lại câu hỏi (chat engine)	Người dùng hỏi tiếp, câu hỏi phụ thuộc ngữ cảnh	Câu hỏi "còn cái kia thì sao?" được hiểu đúng nhờ lịch sử
Reranking	Truy hồi ban đầu lấy đúng đoạn nhưng xếp hạng chưa tốt	Bảng so sánh vị trí của đoạn đúng trước và sau khi rerank
Cập nhật tăng dần	Tài liệu được thêm hoặc sửa thường xuyên	Thêm một tài liệu mới mà không phải dựng lại toàn bộ index

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
So sánh tham số có số liệu	Thay đổi `chunk_size`, `overlap`, `top_k`, hoặc mô hình embedding trên cùng bộ câu hỏi; báo cáo hit rate, MRR và giải thích vì sao kết quả đổi.
Truy hồi kết hợp (vector + từ khóa) và rerank	Kết hợp BM25 với vector, sau đó rerank; so sánh với chỉ dùng vector trên các câu hỏi có từ chuyên ngành.
Câu hỏi nhiều bước	`SubQuestionQueryEngine` hoặc cách tương đương cho câu hỏi cần nhiều nguồn; chứng minh bằng ví dụ mà truy hồi đơn giản thất bại.
Đánh giá có kiểm soát	Sinh bộ câu hỏi bằng LLM nhưng phải duyệt tay một phần; nêu rõ rủi ro đánh giá bằng LLM tự chấm và cách giảm thiểu.
Quan sát và đo chi phí	Ghi lại độ trễ từng khâu (truy hồi, sinh), số token, chi phí mỗi câu hỏi; chỉ ra khâu chậm nhất.
Tác tử (Workflows/agent) dùng query engine như một công cụ	Chỉ nên làm khi bài toán thật sự có nhiều bước hoặc nhiều công cụ; so sánh với truy hồi thường.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ pipeline thật của đồ án, khớp với code đã nộp.
- Với một số câu hỏi: danh sách top-k node được truy hồi (nội dung, điểm, nguồn), không chỉ câu trả lời cuối cùng.
- Bộ câu hỏi kiểm thử nhỏ kèm bảng kết quả truy hồi, câu trả lời và nguồn; trong đó có câu ngoài tài liệu.
- Nêu rõ cấu hình đã dùng (mô hình, chunk, top-k) để nhóm khác chạy lại được.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Dán nguyên tài liệu dài vào prompt rồi hỏi mô hình: đó không phải RAG.
- Chỉ gọi `VectorStoreIndex.from_documents()` rồi `query_engine.query()` mà không xem node, chunk hay kết quả truy hồi.
- Đánh giá bằng vài câu tự chọn lúc demo rồi kết luận hệ thống "chính xác".
- Chọn miền nghiệp vụ quá khó (ví dụ kết luận pháp lý) khiến nhóm phải học chuyên môn ngoài CNTT nhiều hơn học LlamaIndex.

6. Câu hỏi tự kiểm

- Câu trả lời này được tạo từ những node nào? Chúng đến từ tài liệu nào, trang nào?
- Nếu tăng `chunk_size` hoặc `top_k` thì truy hồi thay đổi ra sao? Có số liệu không?
- Khi câu hỏi không có trong tài liệu, hệ thống làm gì? Làm sao biết nó không bịa?
- Cái gì trong đồ án này là LlamaIndex mà viết thẳng bằng thư viện vector và gọi LLM thì phải tự làm?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn LlamaIndex

- **LangChain**: nhiều tích hợp và linh hoạt hơn, nhưng thường phải tự viết nhiều hơn cho phần truy hồi; LlamaIndex tập trung sẵn cho bài toán dữ liệu riêng.
- **Tự viết** (embedding + FAISS hoặc Chroma + gọi LLM): kiểm soát từng bước, ít phụ thuộc; đổi lại phải tự làm chia đoạn, lập chỉ mục, trích nguồn.
- **Haystack**: cũng là framework RAG, cách tổ chức pipeline khác; ít gặp hơn trong cộng đồng tiếng Việt.

Những trường hợp KHÔNG nên dùng LlamaIndex (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Tài liệu ngắn, vừa đủ trong ngữ cảnh của mô hình và ít thay đổi: đưa thẳng vào prompt đơn giản và rẻ hơn.
- Dữ liệu là bảng có cấu trúc rõ (giá, cấu hình, số lượng): truy vấn SQL hoặc bộ lọc chính xác hơn tìm kiếm ngữ nghĩa.
- Bài toán cốt lõi là điều phối nhiều bước và công cụ (agent): LangGraph hợp hơn là LlamaIndex đơn thuần.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thay `chunk_size` hoặc `top_k` và so sánh truy hồi trên cùng bộ câu hỏi.
- Thêm lọc metadata hoặc trích dẫn nguồn vào hệ thống hiện có.
- Tạo một câu hỏi ngoài tài liệu và chỉnh hệ thống để từ chối có căn cứ.

LangGraph (hồ sơ Agentic)

Ứng dụng AI · Framework điều phối tác tử dạng đồ thị có trạng thái · Python · LangGraph 1.x; nhóm ghi rõ phiên bản và mô hình sử dụng.

> **Trọng tâm của đề tài**
> LangGraph điều phối một quy trình nhiều bước dưới dạng **đồ thị có trạng thái (State, Node, Edge)**. Đề tài được đánh giá ở chỗ nhóm chứng minh được vì sao bài toán cần điều phối nhiều bước, mà không thể chỉ gọi mô hình một lần. Một prompt rồi một mô hình rồi một đầu ra mà gọi là "agent" thì chưa đạt.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của ứng dụng AI

- **Bài toán cần điều phối**: liệt kê các bước của quy trình và vì sao không thể thay bằng một lời gọi mô hình duy nhất.
- **Cấu hình mô hình và công cụ**: dùng mô hình nào, tham số nào, độ trễ và chi phí mỗi lần chạy; công cụ (tool) nào được gọi.
- **Dữ liệu vào/ra và giới hạn**: kiểu dữ liệu đi qua các bước; những trường hợp hệ thống không xử lý được.

B. Cơ chế riêng của LangGraph

- **State**: lược đồ trạng thái chạy xuyên các bước (TypedDict hoặc Pydantic) và cách gộp cập nhật (reducer); trạng thái nào được giữ giữa các bước.
- **Nodes và Edges**: mỗi node là một hàm đọc và ghi state; edge nối các bước; vẽ được sơ đồ đồ thị đúng với code.
- **Nhánh điều kiện hoặc vòng lặp có ý nghĩa**: conditional edge dựa trên đầu ra hoặc ý định; có điều kiện dừng.
- **Công cụ (tool)**: lược đồ, quyền gọi, và cách xử lý khi công cụ lỗi.
- **Chặn vòng lặp vô hạn**: giới hạn số bước (`recursion_limit`) hoặc điều kiện dừng; không cho tác tử lặp hoặc gọi công cụ không giới hạn.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Định tuyến có điều kiện theo ý định (intent)	Nhiều loại yêu cầu cần xử lý khác nhau	Hai lần chạy đi hai nhánh khác nhau, có log
Checkpoint và tiếp tục (resume)	Quy trình dài hoặc cần hỏi tiếp	Dừng giữa chừng và chạy tiếp từ checkpoint đúng trạng thái
Human-in-the-loop (`interrupt`)	Bước nhạy cảm cần người xác nhận	Ca được duyệt và ca bị từ chối
Nhánh song song, subgraph	Có việc độc lập có thể chạy cùng lúc	Hai nhánh chạy song song và kết quả được gộp đúng
Nhiều công cụ và chọn công cụ	Tác tử có nhiều khả năng	Log cho thấy chọn đúng công cụ cho từng loại câu hỏi
Timeout và giới hạn số bước	Luôn nên có	Ca bị chặn khi vượt giới hạn
Tracing/observability (LangSmith, Langfuse…)	Cần truy lại lỗi	Xem được một lần chạy qua các node kèm độ trễ

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Bộ đánh giá cố định	Bộ câu hỏi/nhiệm vụ cố định; thước đo như tỷ lệ chọn đúng công cụ, thành công nhiệm vụ, độ trễ, số token; chạy lại khi đổi prompt hoặc mô hình để phát hiện hồi quy.
So sánh cấu hình	Thay mô hình hoặc cấu trúc đồ thị (một node so với nhiều node); phân tích đánh đổi chất lượng, độ trễ, chi phí.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
An toàn của tác tử	Thử tấn công prompt injection qua nội dung công cụ; giới hạn quyền công cụ; chứng minh biện pháp giảm thiểu.
Phục hồi khi lỗi	Công cụ lỗi hoặc mô hình trả sai định dạng; thử lại có giới hạn, quay về nhánh dự phòng.
Quan sát hệ thống	Lưu mô hình, prompt, phiên bản, độ trễ và lỗi cho mỗi lần chạy để tái hiện.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ đồ thị đúng với code thật.
- Log ít nhất một lần chạy qua nhiều bước; nếu có nhánh thì có ca đi qua hai nhánh khác nhau.
- Nếu dùng công cụ: ca công cụ thành công và ca công cụ lỗi.
- Bảng đánh giá nhỏ (tối thiểu vài chục nhiệm vụ hoặc câu hỏi, có ca khó) và nhận xét lỗi.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Một prompt → một mô hình → một đầu ra mà gọi là "agent".
- Dùng tác tử dựng sẵn của thư viện mà không biết state, bước và công cụ được điều phối ra sao.
- Cho tác tử quyền lặp hoặc gọi công cụ không giới hạn.

6. Câu hỏi tự kiểm

- Vẽ đồ thị của em ra. Nhánh nào chạy khi điều kiện này sai?
- State nào được giữ qua các node?
- Nếu công cụ lỗi hoặc mô hình cứ gọi công cụ lặp lại thì workflow dừng thế nào?
- Vì sao bài toán này cần LangGraph mà không phải một chuỗi (chain) đơn giản?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn LangGraph

- **LangChain (chain, LCEL)**: hợp pipeline tuyến tính; LangGraph hợp khi có vòng lặp, nhánh và trạng thái.
- **LlamaIndex Workflows, CrewAI, AutoGen**: cùng nhóm điều phối tác tử; khác triết lý (sự kiện, vai trò). Nhóm cần so sánh bằng trải nghiệm.
- **Tự viết vòng lặp điều khiển**: kiểm soát tối đa, nhưng phải tự làm lưu trạng thái, resume, quan sát.

Những trường hợp KHÔNG nên dùng LangGraph (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Quy trình tuyến tính cố định (trích xuất, biến đổi, trả lời): một chuỗi hoặc vài lời gọi hàm là đủ.
- Tác vụ hỏi đáp tài liệu thông thường: RAG đơn giản đủ, thêm tác tử chỉ tăng chi phí và độ trễ.
- Yêu cầu hành vi hoàn toàn xác định và kiểm chứng được: logic lập trình thông thường đáng tin hơn mô hình điều hướng.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một nhánh điều kiện vào đồ thị.
- Thêm checkpoint hoặc lịch sử và tiếp tục một luồng.
- Đặt giới hạn số bước hoặc timeout và xử lý lỗi công cụ cho workflow hiện có.

MCP SDK (hồ sơ MCP Server)

Ứng dụng AI · SDK của một giao thức mở · Python hoặc TypeScript SDK · nhóm ghi rõ phiên bản SDK, phiên bản đặc tả MCP và transport (stdio hoặc HTTP).

> **Trọng tâm của đề tài**
> MCP là **giao thức** (không phải framework) để ứng dụng AI kết nối với công cụ và dữ liệu bên ngoài; SDK là thứ nhóm dùng để hiện thực một server. Đề tài được đánh giá ở chỗ server có năng lực rõ, **một client thật gọi được**, lược đồ đầu vào và quyền được kiểm soát. Nhóm làm MCP không phải làm chunking hay embedding trừ khi bài toán cần. Dữ liệu của đề tài (ví dụ thông tin đào tạo) có thể ngoài tầm kiểm soát, nên cần có phương án dữ liệu mẫu từ đầu.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của ứng dụng AI

- **Bài toán và dữ liệu**: server mở ra những năng lực nào, dữ liệu lấy từ đâu, có dữ liệu mẫu dự phòng nếu nguồn thật không truy cập được.
- **Dữ liệu vào/ra và giới hạn**: lược đồ vào và ra của từng công cụ; lỗi nào server trả và trả ra sao.
- **Tích hợp với mô hình**: client dùng mô hình nào để gọi công cụ; cách mô hình quyết định gọi công cụ nào.

B. Cơ chế riêng của MCP

- **Dựng server bằng SDK** và giải thích vòng đời kết nối: khởi tạo, thương lượng khả năng (capability), liệt kê công cụ, gọi công cụ, đóng.
- **Tool**: ít nhất một tool có schema đầu vào và đầu ra rõ; kiểm tra đầu vào; trả lỗi có ý nghĩa.
- **Client thật**: kết nối được với một MCP client thật (một ứng dụng trợ lý AI hoặc công cụ kiểm thử), chứng minh client **khám phá (discover)** và **gọi** được tool.
- **Ranh giới quyền**: tool nào được phép làm gì; không cấp quyền rộng hơn nhu cầu; những secret và dữ liệu server truy cập được.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Resources (dữ liệu chỉ đọc)	Cần cung cấp tài liệu hoặc dữ liệu cho mô hình đọc	Client liệt kê và đọc được resource
Prompts (mẫu prompt)	Muốn chuẩn hóa cách dùng	Client dùng prompt template với tham số
Chọn transport: stdio hoặc HTTP (Streamable HTTP)	Chạy cục bộ hoặc từ xa	Giải thích vì sao chọn; chạy được ở transport đã chọn
Nhiều tool theo nhóm chức năng	Có nhiều năng lực	Tên, mô tả, metadata rõ để mô hình chọn đúng
Lifespan và dọn tài nguyên	Có kết nối CSDL hoặc tệp	Tài nguyên được mở khi khởi động và đóng khi tắt
Timeout, retry, ánh xạ lỗi	Gọi dịch vụ ngoài	Ca timeout và lỗi được trả có ý nghĩa
Xác thực, phân quyền hoặc danh sách cho phép (allowlist)	Server truy cập dữ liệu nhạy cảm	Ca không đủ quyền bị chặn
Ghi log và truy vết lời gọi tool	Cần kiểm tra sau	Mỗi lời gọi có log tham số và kết quả (che dữ liệu nhạy cảm)

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Kiểm thử tự động	Test tool và schema; test tích hợp với client giả; dùng công cụ kiểm tra MCP để gọi thử từng tool.
Đo độ trễ và độ tin cậy	Độ trễ mỗi tool; tỷ lệ mô hình gọi đúng tool trên một bộ câu hỏi; phân loại lỗi (chọn sai tool, truyền sai tham số).

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
An toàn	Thử prompt injection qua nội dung resource hoặc kết quả tool; giới hạn quyền; có bước người dùng xác nhận cho thao tác nhạy cảm.
Transport từ xa và xác thực	Chạy server qua HTTP; xác thực bằng token hoặc OAuth theo đặc tả; thử nhiều client.
Quan sát hệ thống	Log có cấu trúc và tracing cho lời gọi tool; truy lại một lời gọi lỗi.

4. Minh chứng cần có trong báo cáo và repo

- Ảnh hoặc log client nhìn thấy tool/resource và gọi thành công ít nhất hai ca (một thành công, một lỗi).
- Mã schema và handler của tool.
- Mô tả quyền, secret và dữ liệu mà server có thể truy cập.
- Cách chạy lại (cấu hình client, lệnh khởi động server) để nhóm khác thử được.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ viết một REST API rồi gọi nó là MCP.
- Chỉ định nghĩa tool "echo" hoặc "hello" không gắn với ứng dụng.
- Cho mô hình quyền gọi shell, tệp hoặc mạng quá rộng mà không có giới hạn.

6. Câu hỏi tự kiểm

- Tool này khác một endpoint REST bình thường ở cách client và mô hình khám phá và gọi thế nào?
- Nếu mô hình truyền tham số sai schema thì server xử lý ra sao?
- Quyền tối thiểu mà tool này cần là gì?
- MCP là framework hay giao thức? Client biết server của em có tool nào bằng cách nào?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn MCP

- **Function calling trực tiếp trong một ứng dụng**: đơn giản khi chỉ một ứng dụng gọi vài hàm; MCP đáng chọn khi muốn nhiều client, nhiều mô hình dùng chung một server.
- **REST/OpenAPI + tích hợp riêng**: chuẩn hóa tốt cho con người và lập trình; không có cơ chế khám phá năng lực dành cho mô hình.
- **Công cụ trong LangChain/LangGraph**: gắn chặt với một framework; MCP độc lập framework.

Những trường hợp KHÔNG nên dùng MCP (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Chỉ có một ứng dụng nội bộ gọi một vài hàm cố định: function calling thông thường đủ và ít phức tạp hơn.
- Dữ liệu hoặc thao tác rất nhạy cảm mà không thể giới hạn quyền ở mức đủ chặt: rủi ro cho phép mô hình gọi vượt tầm kiểm soát.
- Bài toán là hỏi đáp tài liệu: RAG phù hợp hơn, MCP chỉ là cách kết nối.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một tool mới có schema và đường lỗi.
- Thêm resource chỉ đọc hoặc giới hạn quyền của tool hiện có.
- Giải thích và gỡ lỗi một lời gọi tool thất bại từ log của client.

Ollama (hồ sơ Local LLM)

Ứng dụng AI · Runtime chạy mô hình ngôn ngữ cục bộ · nhóm ghi rõ phần cứng (CPU, RAM, GPU/VRAM), mô hình và bản lượng tử hóa đã dùng.

> **Trọng tâm của đề tài**
> Chiều sâu của đề tài nằm ở **việc chọn và đo mô hình chạy trên phần cứng của nhóm**: bản lượng tử hóa, mức dùng RAM và VRAM, độ trễ, chất lượng so với API thương mại. Với đề tài "Ollama + RAG": công nghệ chính là Ollama; phần RAG là ứng dụng, chỉ cần pipeline chạy được kèm một bảng đánh giá truy hồi nhỏ, không phải làm trọn hồ sơ RAG.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của ứng dụng AI

- **Bài toán và pipeline**: ứng dụng làm gì với mô hình cục bộ (hỏi đáp, tóm tắt, phân loại), luồng dữ liệu từ đầu vào tới đầu ra; vì sao chọn chạy cục bộ thay vì dùng API.
- **Phần cứng và môi trường**: cấu hình máy, cách chạy Ollama (cài trực tiếp hoặc Docker), cách gọi (API cục bộ hoặc thư viện).
- **Đánh giá bằng số**: bộ câu hỏi cố định (gợi ý từ 20 câu) làm chuẩn cho mọi so sánh.

B. Cơ chế riêng của Ollama và mô hình cục bộ

- **Chọn mô hình theo phần cứng**: kích thước mô hình (số tham số), vì sao một mô hình nhất định chạy được trên máy của nhóm và mô hình lớn hơn thì không.
- **Lượng tử hóa**: bản lượng tử hóa (ví dụ 4-bit so với 8-bit) đổi gì: bộ nhớ, tốc độ, chất lượng; nhóm thử ít nhất hai bản.
- **Đo mức dùng tài nguyên**: RAM, VRAM, mức dùng CPU/GPU khi chạy mô hình; có ảnh hoặc log từ công cụ giám sát.
- **Cửa sổ ngữ cảnh (context window)**: giới hạn của mô hình đang dùng; điều gì xảy ra khi đầu vào dài hơn; cách xử lý.
- **Đo độ trễ**: thời gian đến token đầu tiên và tốc độ sinh (token mỗi giây) với đầu vào ngắn và dài.
- **Đánh giá chất lượng và so sánh với API thương mại**: chạy cùng bộ câu hỏi trên mô hình cục bộ và một API (ví dụ Gemini); bảng so sánh chất lượng, độ trễ, chi phí.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Cấu hình `num_ctx`, tham số sinh (temperature, top_p), `Modelfile`	Cần điều chỉnh hành vi mô hình	So sánh đầu ra trước/sau khi đổi; giải thích tham số
Dùng nhiều mô hình cho nhiều tác vụ	Tác vụ nhẹ và nặng khác nhau	Bảng phân công mô hình theo tác vụ và lý do
Streaming đầu ra	Giao diện chat	Thời gian đến token đầu tiên
Phục vụ qua API riêng	Nhiều người dùng hoặc nhiều ứng dụng gọi	Xếp hàng hoặc giới hạn đồng thời; ca quá tải
RAG mức ứng dụng (nếu đề tài có)	Hỏi đáp trên tài liệu	Pipeline chạy được và bảng đánh giá truy hồi nhỏ (có câu hỏi ngoài tài liệu)
Đóng gói bằng Docker	Muốn chạy lại dễ	docker compose khởi động Ollama và ứng dụng

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Ma trận thực nghiệm	Mô hình × bản lượng tử hóa × độ dài ngữ cảnh: bảng RAM/VRAM, độ trễ, token/giây, chất lượng; kết luận nên chọn cấu hình nào cho phần cứng của nhóm.
So sánh chất lượng có kiểm soát	Cùng bộ câu hỏi và tiêu chí chấm giữa mô hình cục bộ và API thương mại; chấm tay một phần; nêu thiên lệch nếu dùng mô hình chấm.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Tối ưu suy luận	Offload lớp lên GPU, số luồng, giảm ngữ cảnh; benchmark trước/sau.
Chịu tải và đồng thời	Nhiều yêu cầu cùng lúc: độ trễ và thông lượng thay đổi ra sao; giới hạn bằng hàng đợi.
An toàn và quyền riêng tư	Chứng minh dữ liệu không rời máy (theo dõi kết nối mạng); thử prompt injection nếu ứng dụng nhận tài liệu ngoài.

4. Minh chứng cần có trong báo cáo và repo

- Bảng cấu hình phần cứng và mô hình (tên, kích thước, bản lượng tử hóa).
- Bảng đo RAM/VRAM, độ trễ (đến token đầu tiên) và tốc độ sinh cho ít nhất hai cấu hình.
- Bảng so sánh chất lượng với một API thương mại trên cùng bộ câu hỏi, kèm nhận xét.
- Một sự cố giả lập (hết bộ nhớ, ngữ cảnh quá dài) kèm cách phát hiện và xử lý.
- Hướng dẫn để nhóm khác chạy lại trong 30-45 phút: có phương án dùng mô hình nhỏ chạy được trên máy RAM thấp, lệnh tải mô hình và dữ liệu mẫu.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ cài Ollama, chạy một mô hình rồi chat, không có số đo nào.
- Không biết mô hình đang dùng bản lượng tử hóa nào và vì sao.
- Kết luận "mô hình cục bộ đủ tốt" mà không so sánh với mô hình khác trên cùng bộ câu hỏi.
- Đề tài có RAG nhưng không có bảng đánh giá truy hồi nào.

6. Câu hỏi tự kiểm

- Vì sao chọn bản lượng tử hóa này mà không phải bản lớn hơn?
- Máy của em dùng bao nhiêu RAM/VRAM khi chạy mô hình? Em đo bằng gì?
- Khi tăng độ dài ngữ cảnh thì độ trễ và bộ nhớ đổi ra sao?
- Chất lượng so với API thương mại kém hơn ở loại câu hỏi nào?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Ollama

- **API thương mại (Gemini, OpenAI, Claude)**: chất lượng cao, không cần phần cứng mạnh; đổi lại chi phí, độ trễ mạng và dữ liệu rời khỏi máy.
- **llama.cpp, LM Studio, vLLM**: llama.cpp là nền của nhiều công cụ và kiểm soát sâu hơn; vLLM hợp phục vụ nhiều người dùng trên GPU; Ollama dễ dùng nhất.
- **Hugging Face Transformers**: linh hoạt, tải mô hình trực tiếp; thường nặng hơn và cần cấu hình nhiều hơn.

Những trường hợp KHÔNG nên dùng Ollama (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Máy yếu (ít RAM, không có GPU) mà cần chất lượng cao hoặc trả lời nhanh: mô hình đủ nhỏ để chạy thường không đủ chất lượng.
- Phục vụ nhiều người dùng đồng thời: một máy chạy Ollama không mở rộng tốt.
- Bài toán cần kiến thức rộng và suy luận mạnh: mô hình cục bộ nhỏ dễ sai hơn nhiều so với mô hình lớn qua API.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Đổi bản lượng tử hóa và đo lại mức dùng bộ nhớ, độ trễ.
- Đổi `num_ctx` và giải thích ảnh hưởng tới bộ nhớ và kết quả.
- Chạy cùng một câu hỏi trên mô hình cục bộ và API và nhận xét khác biệt.

Ultralytics YOLO (hồ sơ Computer Vision)

Ứng dụng AI · Thư viện mô hình thị giác máy tính · Python · Ultralytics YOLO (bản ổn định mới); nhóm ghi rõ phiên bản và kích thước mô hình. Khuyến khích dùng Yolo26

> **Trọng tâm của đề tài**
> Đề tài được đánh giá ở **bộ dữ liệu, chỉ số đánh giá và hiểu cơ chế của một bài toán phát hiện đối tượng**. Chỉ gọi mô hình huấn luyện sẵn rồi vẽ khung bao quanh vài ảnh mẫu trên Internet là chưa đủ. Nhóm cần có dữ liệu của chính nhóm, có số liệu đánh giá và có phần đưa mô hình thành dịch vụ gọi được (theo ngưỡng của hồ sơ Computer Vision trong danh mục).

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của ứng dụng AI

- **Pipeline của đồ án**: dữ liệu → tiền xử lý → suy luận hoặc huấn luyện lại → ngưỡng tin cậy và NMS → đánh giá → đưa thành dịch vụ.
- **Cấu hình và phần cứng**: kích thước mô hình (nano, small…), kích thước ảnh đầu vào, chạy trên CPU hay GPU, độ trễ thực tế.
- **Dữ liệu vào/ra và giới hạn**: điều kiện mà hệ thống làm tốt và làm kém (ánh sáng, góc chụp, đối tượng nhỏ, bị che).

B. Cơ chế riêng của YOLO

- **Bộ dữ liệu**: nguồn, các lớp nhãn, định dạng chú thích (nhãn YOLO dạng `class x y w h` chuẩn hóa), cách chia tập huấn luyện, kiểm định và kiểm tra (hoặc ít nhất một tập đánh giá riêng); nhóm tự gán nhãn hoặc chỉnh sửa tập dữ liệu và biết tập đó có sai lệch gì.
- **Suy luận và đầu ra**: hộp bao (box), lớp (class), độ tin cậy (confidence); mặt nạ nếu bài toán phân đoạn.
- **Chỉ số đánh giá**: precision, recall, mAP (mAP@0.5 và mAP@0.5:0.95) và ý nghĩa của từng chỉ số; không chỉ báo "độ chính xác" chung chung.
- **Ngưỡng tin cậy và NMS**: giải thích vì sao một hộp bị giữ hay bị loại; tăng giảm ngưỡng ảnh hưởng precision và recall thế nào trên dữ liệu của nhóm (có đo, không chỉ thử tay).
- **Đưa thành dịch vụ**: một API đơn giản nhận ảnh trả kết quả, có kiểm tra đầu vào và đo độ trễ.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Tiền xử lý, thay đổi kích thước, tăng cường dữ liệu	Dữ liệu ít hoặc điều kiện đa dạng	So sánh kết quả có và không tăng cường
Fine-tune trên dữ liệu riêng	Mô hình huấn luyện sẵn chưa phù hợp bài toán	Bảng so sánh mô hình gốc và mô hình đã tinh chỉnh trên cùng tập kiểm tra
Phục vụ qua API (FastAPI)	Muốn tách mô hình khỏi giao diện	Request và response mẫu; độ trễ mỗi request
Xuất ONNX hoặc định dạng khác	Cần tối ưu triển khai	Độ trễ và kết quả khác nhau giữa bản gốc và bản xuất
Theo vết đối tượng (tracking)	Video, đếm hoặc theo dõi	Mã định danh ổn định qua các khung hình
Benchmark độ trễ và FPS trên CPU/GPU hoặc thiết bị đích	Cần chạy thời gian thực	Bảng độ trễ và FPS theo cấu hình
Phân tích lỗi theo lớp, kích thước, ánh sáng	Luôn nên làm	Một nhóm lỗi cụ thể (false positive, false negative) và nguyên nhân
Với bài toán biển số xe: tách riêng khâu nhận dạng ký tự (OCR)	Ứng dụng đọc biển số	Đo riêng độ chính xác phát hiện biển và độ chính xác đọc ký tự; xử lý biển hai dòng, góc chụp nghiêng

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Thực nghiệm so sánh có kiểm soát	Thay kích thước mô hình, kích thước ảnh, mức tăng cường; bảng precision, recall, mAP, độ trễ; kết luận đánh đổi chất lượng và tốc độ.
Đường cong và ngưỡng tối ưu	Vẽ đường cong precision–recall hoặc F1 theo ngưỡng; chọn ngưỡng cho ứng dụng và giải thích.
Phân tích lỗi sâu	Ma trận nhầm lẫn; thống kê lỗi theo kích thước đối tượng và điều kiện chụp; đề xuất và thử một cải tiến dữ liệu.
Tối ưu suy luận	Lượng tử hóa, ONNX Runtime, batch; benchmark CPU và GPU; số liệu trước/sau.
Kiểm thử và quan sát	Bộ ảnh kiểm thử hồi quy cố định; log độ trễ và độ tin cậy cho mỗi request của dịch vụ.

4. Minh chứng cần có trong báo cáo và repo

- Bộ dữ liệu của nhóm (nguồn, lớp nhãn, số lượng, cách chia tập), kèm vài ví dụ nhãn.
- Bảng precision, recall, mAP trên tập đánh giá, kèm vài ví dụ đúng và sai.
- Kết quả đo ảnh hưởng của ngưỡng tin cậy hoặc NMS trên dữ liệu của nhóm.
- Nếu triển khai dịch vụ: request/response mẫu và độ trễ.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ dùng mô hình huấn luyện sẵn trên vài ảnh mẫu từ Internet rồi chụp màn hình các hộp bao.
- Không có dữ liệu chuẩn (ground truth) nhưng vẫn kết luận "độ chính xác cao".
- Chỉ chỉnh ngưỡng tin cậy bằng tay mà không đo ảnh hưởng.
- Báo mAP nhưng không biết tính trên tập nào.

6. Câu hỏi tự kiểm

- mAP của nhóm được tính trên tập nào?
- Tăng ngưỡng tin cậy thì precision và recall thay đổi thế nào trong dữ liệu này?
- Một lỗi false positive cụ thể đến từ đâu?
- NMS làm gì? Nếu tắt đi thì kết quả khác ra sao?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn YOLO

- **Các mô hình phát hiện khác (Faster R-CNN, RT-DETR, SSD)**: YOLO nổi bật ở tốc độ và công cụ huấn luyện, xuất mô hình tiện; nhóm cần nêu so sánh dựa trên thử nghiệm hoặc tài liệu.
- **OpenCV cổ điển hoặc MediaPipe**: không cần dữ liệu huấn luyện cho một số bài toán đặc thù (khuôn mặt, tư thế); YOLO hợp khi đối tượng tự định nghĩa.
- **API thị giác trên đám mây**: nhanh để thử, nhưng chi phí, độ trễ và quyền riêng tư; mô hình tự huấn luyện kiểm soát được dữ liệu.

Những trường hợp KHÔNG nên dùng YOLO (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Chỉ cần biết ảnh thuộc loại nào (phân loại), không cần vị trí: mô hình phân loại ảnh đơn giản hơn.
- Dữ liệu rất ít và điều kiện thực tế quá khác dữ liệu huấn luyện: mô hình dễ kém khi triển khai.
- Sản phẩm thương mại đóng nguồn: thư viện Ultralytics dùng giấy phép AGPL-3.0, cần xem xét giấy phép trước khi dùng.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một lớp hoặc dữ liệu mới và chạy lại đánh giá.
- Đổi ngưỡng hoặc cấu hình NMS và giải thích thay đổi của đầu ra.
- Đo độ trễ hoặc FPS ở hai cấu hình nếu nhóm đã làm phần hiệu năng.

faster-whisper (hồ sơ Speech-to-Text)

Ứng dụng AI · Thư viện nhận dạng tiếng nói · Python · faster-whisper (chạy mô hình Whisper trên CTranslate2); nhóm ghi rõ kích thước mô hình, thiết bị và `compute_type`.

> **Trọng tâm của đề tài**
> Âm thanh, video, phụ đề và giao diện chỉ là bối cảnh. Đề tài được đánh giá ở **pipeline nhận dạng tiếng nói (ASR) và các đánh đổi tốc độ – chất lượng – tài nguyên**. Chỉ gọi `transcribe()` rồi in chữ ra màn hình là chưa đạt.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của ứng dụng AI

- **Pipeline của đồ án**: tệp âm thanh/video → chuẩn hóa audio → nhận dạng → xử lý segment → xuất kết quả (văn bản, SRT, VTT).
- **Cấu hình và tài nguyên**: kích thước mô hình, CPU hay GPU, `compute_type`, thời gian xử lý thực tế và bộ nhớ.
- **Dữ liệu vào/ra và giới hạn**: loại âm thanh làm tốt và làm kém (nhiễu, giọng địa phương, tên riêng, nhiều người nói).

B. Cơ chế riêng của faster-whisper

- **Đầu vào audio/video**: trích âm thanh và chuẩn hóa (tần số lấy mẫu, kênh) cho ASR; tránh mã hóa lại có tổn hao không cần thiết.
- **Tải và chạy mô hình**: vai trò của kích thước mô hình, thiết bị (`device`) và `compute_type` (ví dụ float16, int8) với tốc độ, chất lượng và bộ nhớ.
- **Transcript theo segment**: mỗi segment có thời điểm bắt đầu, kết thúc và văn bản; giải thích cấu trúc đầu ra.
- **Ngôn ngữ và tham số giải mã**: chỉ định hoặc phát hiện ngôn ngữ; ít nhất một tham số giải mã (ví dụ `beam_size`) và ảnh hưởng của nó tới tốc độ và chất lượng.
- **Lỗi ASR thường gặp**: nhiễu, tên riêng, giọng nói, tốc độ nói, âm thanh kém; nhận ra trong kết quả của chính nhóm.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Mốc thời gian theo từ (word-level timestamps)	Cần căn phụ đề chính xác	Xuất JSON có thời điểm từng từ; so với mốc segment
VAD (phát hiện giọng nói)	Audio có nhiều khoảng lặng	So sánh có và không VAD trên cùng tệp: thời gian, lỗi bịa chữ ở khoảng lặng
Chia nhỏ (chunking) audio dài và ghép kết quả	Tệp dài hàng giờ	Tệp dài chạy hết, mốc thời gian không lệch ở chỗ nối
Tách và xuất phụ đề (SRT/VTT/TXT)	Mục tiêu là phụ đề hoặc biên bản	Tệp SRT mở trong trình phát thấy đúng thời điểm
Phát hiện ngôn ngữ	Nhiều ngôn ngữ	Ca nhận đúng và ca nhận sai
Xử lý hàng loạt (batch)	Nhiều tệp	Thời gian tổng và cách xử lý lỗi từng tệp
Phục vụ qua API	Muốn tách mô hình khỏi giao diện	Request và response mẫu; xử lý nhiều yêu cầu đồng thời

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Đo chất lượng WER/CER	Có bản chép chuẩn (ground truth) cho một mẫu; chuẩn hóa văn bản trước khi tính (chữ hoa, dấu câu); báo WER hoặc CER và phân tích loại lỗi.
Bảng so sánh cấu hình	Kích thước mô hình × `compute_type` × thiết bị; thời gian xử lý, hệ số thời gian thực (RTF), bộ nhớ, WER; kết luận đánh đổi.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Tối ưu cho tệp dài	VAD, xử lý theo lô, giới hạn bộ nhớ; thời gian trước/sau.
Hậu xử lý	Chuẩn hóa tên riêng bằng từ điển, thêm dấu câu; đo ảnh hưởng tới WER hoặc chất lượng phụ đề.
Nhiều người nói	Ghép với phân tách người nói (diarization) bằng công cụ khác, nêu rõ faster-whisper tự nó không làm việc này.

4. Minh chứng cần có trong báo cáo và repo

- Ít nhất một tệp audio hoặc video dài từ 10 đến 60 phút chạy được từ đầu đến cuối.
- Transcript có mốc thời gian; nếu làm phụ đề thì tệp SRT/VTT đúng thời điểm cơ bản.
- Ít nhất một phép đo: thời gian xử lý, RTF, WER/CER trên mẫu có ground truth, hoặc so sánh hai cấu hình.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ gọi `transcribe()` và in chữ ra màn hình.
- Không giải thích được mô hình, thiết bị, `compute_type` hoặc mốc thời gian.
- Chỉ làm giao diện tải lên/tải xuống mà không đo hay phân tích ASR.

6. Câu hỏi tự kiểm

- Đổi kích thước mô hình hoặc `compute_type` thì đổi điều gì?
- Mốc thời gian theo segment khác theo từ ở đâu, và ứng dụng của em cần loại nào?
- Khoảng lặng dài ảnh hưởng pipeline thế nào? VAD giúp gì?
- WER của em tính trên mẫu nào và chuẩn hóa văn bản ra sao?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn faster-whisper

- **Whisper gốc (openai-whisper)**: cùng mô hình nhưng chậm hơn và tốn bộ nhớ hơn; faster-whisper nhanh hơn nhờ CTranslate2.
- **whisper.cpp**: nhẹ, chạy tốt trên máy cá nhân hoặc thiết bị; faster-whisper thuận tiện hơn trong hệ Python.
- **API nhận dạng tiếng nói trên đám mây**: ít phải cài đặt và có thể tốt cho nhiều ngôn ngữ; đổi lại chi phí, độ trễ mạng và quyền riêng tư.

Những trường hợp KHÔNG nên dùng faster-whisper (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Cần nhận dạng thời gian thực với độ trễ rất thấp (hội thoại trực tiếp): kiến trúc xử lý theo luồng chuyên dụng phù hợp hơn xử lý theo tệp.
- Chất lượng tiếng Việt chưa đạt cho một miền chuyên ngành hoặc giọng địa phương: cần tinh chỉnh hoặc nguồn khác.
- Cần phân tách người nói ngay trong cùng công cụ: cần ghép thêm thành phần khác.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm mốc thời gian theo từ và xuất JSON hoặc SRT tương ứng.
- Bật hoặc tinh chỉnh VAD cho một tệp có nhiều khoảng lặng và so sánh.
- Đổi hai cấu hình mô hình/`compute_type`, đo lại thời gian và giải thích đánh đổi.

Gemini API (SDK mô hình)

Ứng dụng AI · SDK gọi mô hình · Google Gen AI SDK (Python hoặc JavaScript) · nhóm ghi rõ SDK, tên mô hình và phiên bản đang dùng.

> **Trọng tâm của đề tài**
> Gọi một API rồi in chữ ra giao diện **chưa phải là một đề tài đủ sâu**. Để ngang với các nhóm khác, nhóm cần có **bộ kiểm thử cố định, so sánh hai cấu hình có số liệu, ràng buộc đầu ra theo lược đồ, xử lý lỗi, và thử tấn công prompt**; đồng thời nên ghép với một hồ sơ rõ hơn (RAG, tác tử, đa phương thức) hoặc làm nghiên cứu so sánh và đánh giá đủ sâu. Nhóm chọn hồ sơ ghép ngay từ đầu để biết quy trình bắt buộc đi kèm.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của ứng dụng AI

- **Bài toán và pipeline**: ứng dụng dùng mô hình để làm việc gì, đầu vào và đầu ra là gì, các bước trước và sau khi gọi mô hình; vì sao cần mô hình thay vì luật lập trình.
- **Cấu hình mô hình**: bảng cấu hình thực tế (tên mô hình, `temperature`, giới hạn token đầu ra, system instruction); vì sao chọn mô hình này mà không phải mô hình nhỏ hơn và rẻ hơn.
- **Đánh giá bằng số**: bộ kiểm thử cố định (gợi ý từ 20 ca, có ca khó và ca ngoài phạm vi) và tiêu chí chấm; không kết luận "tốt" chỉ từ vài lần thử khi demo.

B. Cơ chế riêng khi dùng SDK mô hình

- **Gọi API đúng và an toàn**: dùng SDK chính thức; khóa API lưu bằng biến môi trường hoặc kho bí mật, **tuyệt đối không để trong mã nguồn hay repo**; có `.env.example`.
- **Đặt tham số có chủ đích**: thay `temperature`, giới hạn token, và giải thích tác động lên chính tác vụ của nhóm (có thử và có kết quả).
- **Theo dõi token, độ trễ và chi phí**: log số token vào/ra và thời gian cho từng request; ước lượng chi phí mỗi request và mỗi ngày dùng.
- **Thiết kế prompt có cấu trúc**: tách system instruction, ví dụ mẫu và dữ liệu người dùng; kiểm tra đầu ra thay vì nối chuỗi tùy tiện.
- **Xử lý lỗi**: timeout, giới hạn tần suất (lỗi 429), lỗi máy chủ, nội dung bị chặn; ứng dụng phản ứng thế nào, thử lại ra sao.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Streaming	Giao diện cần hiện chữ dần	Đo thời gian đến token đầu tiên so với chờ đủ
Structured output (đầu ra theo JSON schema) và kiểm tra bằng Pydantic/Zod	Cần đầu ra có cấu trúc để xử lý tiếp	Tỷ lệ đầu ra hợp lệ trên bộ kiểm thử; ca sai schema và cách xử lý
Function/tool calling	Mô hình cần gọi hàm của ứng dụng	Ca gọi đúng hàm, ca tham số sai, ca hàm lỗi
Đa phương thức (ảnh, âm thanh, tài liệu)	Đầu vào không chỉ là chữ	Bộ ca có ảnh và đánh giá riêng
Cache theo câu hỏi và giới hạn tần suất	Nhiều câu hỏi lặp lại	Tỷ lệ trúng cache và chi phí tiết kiệm được
Retry, backoff, mô hình dự phòng (fallback)	Muốn chịu lỗi	Giả lập lỗi 429/timeout và hệ thống tự phục hồi
Quản lý prompt theo phiên bản và kiểm thử hồi quy	Prompt thay đổi nhiều	Chạy lại bộ kiểm thử sau mỗi thay đổi và so sánh

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
So sánh mô hình và cấu hình	Cùng bộ kiểm thử với hai mô hình (ví dụ bản nhanh và bản mạnh) hoặc hai prompt; bảng chất lượng, độ trễ, token, chi phí; kết luận đánh đổi.
An toàn và chống tấn công prompt	Thử prompt injection và rò rỉ thông tin hệ thống; có biện pháp giảm thiểu và kết quả thử lại.
Tối ưu chi phí	Rút gọn ngữ cảnh, dùng context caching hoặc batch khi phù hợp; chi phí trước/sau.
Đánh giá có kiểm soát	Dùng mô hình làm giám khảo (LLM-as-judge) nhưng đối chiếu với chấm tay một phần; nêu thiên lệch của cách đánh giá này.
Ghép hồ sơ chuyên sâu	Thêm RAG (đánh giá truy hồi), hoặc tác tử có công cụ, hoặc đa phương thức với đánh giá riêng; làm theo quy trình bắt buộc của hồ sơ đã chọn.
Quan sát hệ thống	Log có cấu trúc, tracing (Langfuse hoặc tương đương) và bảng lỗi theo loại.

4. Minh chứng cần có trong báo cáo và repo

- Bảng cấu hình mô hình và prompt thực tế, và ít nhất một so sánh hai cấu hình hoặc hai mô hình.
- Log token, độ trễ và chi phí cho một số request.
- Bộ kiểm thử cố định kèm bảng kết quả và nhận xét lỗi.
- Nếu dùng structured output hoặc tool calling: ca sai schema hoặc ca công cụ lỗi.
- Một sự cố giả lập (hết hạn mức, timeout) kèm cách phát hiện và sửa.
- Hướng dẫn để nhóm khác chạy lại trong 30-45 phút: cách tạo khóa API riêng (không dùng chung khóa của nhóm), cấu hình `.env.example`, bộ kiểm thử mẫu.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới file hoặc commit cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Chỉ gọi một API rồi in chữ ra giao diện.
- Không biết đang dùng mô hình nào, tham số nào, chi phí bao nhiêu.
- Để khóa API trong mã nguồn hoặc repo.
- Đánh giá bằng vài câu tự chọn lúc demo rồi kết luận "chính xác".

6. Câu hỏi tự kiểm

- Tại sao chọn mô hình này thay vì mô hình nhỏ hơn hoặc rẻ hơn?
- Tăng `temperature` thì đầu ra của em đổi thế nào? Em đã thử chưa?
- Nếu API timeout hoặc bị giới hạn tần suất thì ứng dụng xử lý thế nào?
- Điều gì xảy ra nếu người dùng cố ép mô hình bỏ qua chỉ dẫn hệ thống?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Gemini API

- **OpenAI, Claude (API thương mại khác)**: cùng loại SDK; khác nhau về chất lượng từng tác vụ, giá, độ trễ, tính năng. Nhóm cần so sánh bằng thử nghiệm của mình.
- **Chạy mô hình cục bộ (Ollama)**: kiểm soát dữ liệu và không tính phí theo lượt gọi; đổi lại phần cứng và chất lượng thường thấp hơn.
- **Dịch vụ dựng sẵn (chatbot builder)**: nhanh, nhưng không cho thấy hiểu biết kỹ thuật về mô hình.

Những trường hợp KHÔNG nên dùng Gemini API (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Bài toán xác định có luật rõ (kiểm tra định dạng, tính toán): code thông thường chính xác, nhanh và rẻ hơn gọi mô hình.
- Dữ liệu nhạy cảm không được gửi ra dịch vụ bên ngoài: cần mô hình chạy cục bộ.
- Cần câu trả lời đúng tuyệt đối và kiểm chứng được: mô hình có thể bịa; phải có bước kiểm chứng hoặc không dùng.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm structured output cho một phản hồi đang là văn bản tự do.
- Thêm retry/backoff hoặc mô hình dự phòng cho một lỗi API.
- Đo độ trễ và chi phí của hai mô hình nếu nhóm đã làm tối ưu.

n8n

Tự động hóa · Nền tảng workflow mã nguồn mở, tự triển khai được · n8n bản mới ổn định; nhóm ghi rõ phiên bản và cách triển khai.

> **Trọng tâm của đề tài**
> n8n là nền tảng duy nhất trong nhóm low-code tự triển khai được bằng Docker. Chiều sâu đến từ **trigger, ánh xạ dữ liệu giữa các node, xử lý lỗi và tích hợp thật**, không phải số node kéo thả. Nhóm không bị ép viết nhiều code, nhưng phải đạt cùng độ sâu minh chứng như các nhóm khác: có bảng kiểm thử, có đo, có sự cố giả lập.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của tự động hóa

- **Trigger**: quy trình được kích hoạt bởi sự kiện (webhook), lịch hoặc thăm dò (polling); vì sao chọn loại này cho bài toán.
- **Rẽ nhánh theo điều kiện**: node điều kiện (`IF`, `Switch`); có ít nhất hai nhánh khác nhau đều được chạy trong kiểm thử.
- **Biến đổi dữ liệu giữa các bước**: chuẩn hóa định dạng, xử lý trường thiếu; biểu thức của n8n và cách đọc dữ liệu từ node trước.
- **Xử lý lỗi**: phát hiện lỗi, ghi nhận, và có hành vi xác định khi một bước thất bại (không để workflow chết im lặng).
- **Tích hợp từ hai hệ thống trở lên**: ví dụ biểu mẫu, bảng tính, cơ sở dữ liệu, email, chat hoặc API ngoài.

B. Cơ chế riêng của n8n

- **Webhook**: nhận request vào workflow, có URL kiểm thử và URL chạy thật; đọc được dữ liệu vào (headers, body).
- **Dữ liệu dạng "item" giữa các node**: mỗi node nhận và trả mảng item; ánh xạ trường giữa các node; hiểu vì sao một node chạy nhiều lần khi nhận nhiều item.
- **Error workflow**: một workflow riêng được gọi khi workflow chính lỗi; nội dung nó làm (thông báo, ghi log).
- **Tự triển khai bằng Docker**: chạy n8n bằng `docker compose`, dữ liệu lưu bền (volume hoặc CSDL ngoài), biến môi trường (khóa mã hóa, URL webhook).
- **Credential**: thông tin xác thực lưu trong kho credential của n8n, không dán trực tiếp vào node hay xuất ra tệp công khai.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Node Code (JavaScript hoặc Python)	Logic không có node dựng sẵn	Đoạn code tự viết, đầu vào và đầu ra; ca lỗi
Sub-workflow	Có phần dùng lại ở nhiều workflow	Workflow con được gọi từ ít nhất hai nơi
Cơ chế thử lại (retry) của node	Dịch vụ ngoài thỉnh thoảng lỗi	Giả lập lỗi tạm thời và thấy thử lại thành công
Đọc và phân tích execution log	Luôn nên làm	Chỉ ra một lần chạy lỗi ở node thứ N và cách tìm lại nó
Xác thực chữ ký webhook	Nhận dữ liệu từ dịch vụ bên thứ ba	Request giả mạo bị từ chối
Node AI (LLM, bộ nhớ, công cụ)	Tích hợp mô hình ngôn ngữ vào luồng	Ca đầu ra sai định dạng và cách xử lý
Lịch và thao tác theo đợt (batch)	Xử lý hàng loạt định kỳ	Chạy đúng lịch; xử lý số lượng lớn không quá giới hạn

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Chống xử lý trùng (idempotency)	Webhook nhận trùng một sự kiện hai lần; chứng minh workflow không tạo bản ghi trùng (khóa duy nhất, kiểm tra trước).

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Độ tin cậy và khả năng mở rộng	Chế độ queue với Redis và nhiều worker (nếu phù hợp); đo thời gian xử lý khi tăng tải trước/sau.
Kiểm thử workflow	Bộ ca kiểm thử (khoảng 15-20 ca, gồm ca lỗi) chạy lặp lại được bằng dữ liệu mẫu; bảng kết quả.
Quản lý phiên bản và triển khai	Xuất workflow ra `.json` đưa vào Git; mô tả cách nhập lại ở máy khác; biến môi trường theo môi trường.
Bảo mật	Webhook có xác thực, quyền tối thiểu cho credential, không lộ bí mật trong log và execution.
Quan sát hệ thống	Thông báo khi workflow lỗi; thống kê tỷ lệ thành công và thời gian chạy từ execution history.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ workflow (ảnh) và mô tả trigger, các nhánh, các hệ thống được tích hợp.
- Bảng ca kiểm thử (khoảng 15-20 ca, gồm ca lỗi) và kết quả.
- Một lỗi giả lập ở một node cụ thể, kèm execution log và cách workflow phản ứng (error workflow, thông báo).
- Tệp `docker-compose.yml` và hướng dẫn chạy lại; workflow xuất ra `.json`.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới workflow hoặc node cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Một workflow thẳng một đường vài node, không rẽ nhánh, không xử lý lỗi.
- Chỉ dùng n8n Cloud hoặc bản có sẵn, không tự triển khai bằng Docker.
- Dán khóa API trực tiếp vào node hoặc chia sẻ trong tệp xuất workflow.
- Không có execution log nào cho thấy workflow đã chạy.

6. Câu hỏi tự kiểm

- Workflow chạy lỗi ở node thứ tư. Em tìm lại lần chạy đó bằng cách nào?
- Webhook nhận cùng một sự kiện hai lần thì chuyện gì xảy ra?
- Dữ liệu từ node này sang node kia được ánh xạ thế nào? Nếu trường thiếu thì sao?
- Credential được lưu ở đâu và vì sao không nằm trong tệp workflow?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn n8n

- **Zapier, Make**: dễ dùng, dịch vụ đám mây; n8n tự triển khai được và có node code, hợp khi cần kiểm soát dữ liệu.
- **Power Automate**: tích hợp sâu hệ Microsoft 365; n8n độc lập nền tảng.
- **Tự viết bằng code (cron + script, hàng đợi)**: linh hoạt tối đa nhưng phải tự làm trigger, retry, giám sát; n8n cho thấy trực quan nhưng bị ràng buộc bởi mô hình node.

Những trường hợp KHÔNG nên dùng n8n (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Logic nghiệp vụ rất phức tạp hoặc nhiều tính toán nặng: viết bằng code trong dịch vụ riêng dễ kiểm thử và bảo trì hơn.
- Cần độ trễ rất thấp hoặc thông lượng rất cao: n8n không phải bộ xử lý dòng dữ liệu.
- Cần kiểm thử tự động và quản lý phiên bản chặt như phần mềm: workflow dạng đồ họa khó so sánh khác biệt hơn mã.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một nhánh điều kiện hoặc node code vào workflow có sẵn và chứng minh hai nhánh chạy.
- Thêm error workflow hoặc thông báo cho một lỗi giả lập.
- Tìm trong execution history một lần chạy lỗi và giải thích nguyên nhân.

Power Apps + Power Automate

Low-code · Nền tảng Microsoft Power Platform · Power Apps (canvas app hoặc model-driven) + Power Automate; nhóm ghi rõ loại ứng dụng, nguồn dữ liệu (Dataverse, SharePoint, Excel) và loại tài khoản.

> **Trọng tâm của đề tài**
> Low-code không bị đánh giá thấp vì ít mã nguồn. Chiều sâu phải đến từ **Power Fx, mô hình dữ liệu có quan hệ, delegation, phân quyền theo vai trò và flow có điều kiện, phê duyệt, xử lý lỗi**. Để ngang với các nhóm viết code, nhóm nộp bộ ca kiểm thử (đúng và sai), dữ liệu đủ lớn để thấy giới hạn của nền tảng, và một sự cố giả lập có cách phát hiện và sửa.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của tự động hóa low-code

- **Trigger** của flow: sự kiện (ví dụ khi tạo bản ghi), lịch hoặc thao tác từ ứng dụng; vì sao chọn loại này.
- **Rẽ nhánh theo điều kiện** trong flow (điều kiện, `switch`) và có ca kiểm thử đi qua từng nhánh.
- **Biến đổi dữ liệu giữa các bước**: chuẩn hóa định dạng, xử lý trường thiếu.
- **Xử lý lỗi**: có hành vi xác định khi một bước thất bại (ví dụ thiết lập "chạy sau khi lỗi", nhóm bước dùng như try/catch, thông báo cho người phụ trách).
- **Tích hợp từ hai hệ thống trở lên**: ví dụ ứng dụng với SharePoint, Outlook/Teams, Excel, một API ngoài qua connector.

B. Cơ chế riêng của Power Platform

- **Power Fx**: nhiều công thức do nhóm tự viết (`Filter`, `LookUp`, `Patch`, `If`, `Collect`, ...) có ý nghĩa nghiệp vụ; hiểu công thức nào chạy ở phía client, công thức nào được đẩy xuống nguồn dữ liệu.
- **Nguồn dữ liệu và quan hệ**: Dataverse (bảng, cột, quan hệ) hoặc SharePoint list/Excel; sơ đồ bảng và giải thích một quan hệ thật.
- **Delegation**: khái niệm giao việc lọc/sắp xếp cho nguồn dữ liệu; hàm nào không được delegate với nguồn của nhóm; hệ quả (kết quả thiếu khi dữ liệu vượt giới hạn số dòng mặc định) và cách xử lý.
- **Connector và trigger/action của Power Automate**: flow nào gọi từ ứng dụng, flow nào chạy nền; truyền tham số giữa ứng dụng và flow.
- **Phân quyền cơ bản**: người dùng nào nhìn thấy và sửa được gì; không chỉ ẩn nút trên giao diện.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Quy trình phê duyệt (Approvals)	Có luồng duyệt nhiều bước	Ca được duyệt, từ chối và quá hạn
Phân quyền theo vai trò (bảo mật hàng/cột của Dataverse, nhóm bảo mật)	Nhiều loại người dùng	Người dùng A không thấy dữ liệu của B: chứng minh ở cả dữ liệu, không chỉ ở màn hình
Delegation: thiết kế để truy vấn lớn vẫn đúng	Dữ liệu từ vài nghìn dòng trở lên	Cùng một bảng lớn: công thức delegate được cho kết quả đúng, công thức không delegate cho kết quả thiếu
Environment và solution, ALM	Muốn quản lý phiên bản và triển khai	Xuất solution ra tệp `.zip` và nhập lại được ở môi trường khác
Xử lý lỗi trong flow (scope, run after)	Flow gọi dịch vụ ngoài	Giả lập lỗi và thấy nhánh xử lý lỗi chạy
Kiểm tra dữ liệu đầu vào (validation) trên ứng dụng	Biểu mẫu nhập liệu	Ca dữ liệu sai bị chặn và thông báo đúng
Component hoặc thư viện component dùng lại	Nhiều màn hình giống nhau	Component dùng lại ở nhiều màn hình

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Dữ liệu lớn và hiệu năng	Bảng nhiều nghìn dòng; đo thời gian tải màn hình trước/sau khi sửa công thức để delegate hoặc dùng chỉ mục; ghi lại giới hạn gặp phải.
Bộ ca kiểm thử có hệ thống	Khoảng 15-20 ca gồm ca lỗi; có thể dùng Test Studio hoặc bảng ca chạy tay có ảnh; kết quả đạt hoặc không đạt.
Quản lý vòng đời ứng dụng	Solution với nhiều môi trường (phát triển và kiểm thử), biến môi trường, và quy trình nhập xuất.
Phân quyền sâu	Bảo mật theo hàng bằng vai trò của Dataverse; thử đăng nhập bằng hai tài khoản khác vai trò và so sánh.
Quan sát và giám sát	Lịch sử chạy flow, thống kê tỷ lệ lỗi, thông báo khi flow thất bại.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ bảng (và quan hệ) cùng ảnh các màn hình, và sơ đồ các flow.
- Nhiều công thức Power Fx tự viết kèm ảnh tình huống đúng và sai.
- Một tình huống delegation: bảng đủ lớn, công thức không delegate được, và cách xử lý.
- Một sự cố giả lập trong flow kèm lịch sử chạy và cách xử lý.
- Tệp xuất solution (`.zip`) hoặc ảnh chụp đầy đủ, kèm hướng dẫn tạo môi trường dùng thử cho nhóm khác.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới màn hình, công thức hoặc flow cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Ứng dụng vài màn hình CRUD dùng công thức mặc định, gần như không có Power Fx tự viết.
- Ẩn nút trên giao diện rồi coi là đã phân quyền trong khi dữ liệu vẫn tải về đầy đủ.
- Flow một đường thẳng, không có điều kiện, không xử lý lỗi.
- Chỉ dùng dữ liệu vài dòng nên không bao giờ gặp giới hạn của nền tảng.

6. Câu hỏi tự kiểm

- Bảng có 50.000 dòng, màn hình của em lọc có đúng không? Delegation ảnh hưởng thế nào?
- Công thức này chạy ở đâu, ứng dụng hay nguồn dữ liệu?
- Flow lỗi ở bước thứ ba thì ai biết và hệ thống làm gì?
- Người dùng A không được thấy dữ liệu của B. Em chặn ở đâu, và chặn ở giao diện có đủ không?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn Power Apps

- **Google AppSheet**: cũng no-code dựa trên bảng tính/CSDL; hợp hệ Google. Power Apps hợp doanh nghiệp đã dùng Microsoft 365.
- **Ứng dụng web viết bằng code**: linh hoạt hoàn toàn, nhưng tốn thời gian hơn cho ứng dụng nội bộ đơn giản.
- **n8n, Zapier**: mạnh về tự động hóa; không dựng giao diện ứng dụng như Power Apps.

Những trường hợp KHÔNG nên dùng Power Apps (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Ứng dụng công khai hướng người dùng đại trà với giao diện tùy biến cao: nền tảng low-code bị giới hạn về giao diện và chi phí giấy phép.
- Cần logic hoặc hiệu năng đặc thù mà công thức khai báo không đáp ứng được: viết bằng code.
- Nhóm không có tài khoản và giấy phép phù hợp: một số connector và Dataverse cần giấy phép trả phí.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một công thức Power Fx lọc hoặc kiểm tra có điều kiện vào màn hình có sẵn.
- Thêm một nhánh điều kiện hoặc xử lý lỗi vào flow hiện có.
- Giải thích một cảnh báo delegation và sửa nó nếu có thể.

Google AppSheet (kèm hệ sinh thái Google)

No-code · Google AppSheet là công nghệ chính; Looker Studio, Apps Script, Firebase là công nghệ hỗ trợ (Firebase không đăng ký làm công nghệ chính).

> **Trọng tâm của đề tài**
> Đề tài "hệ sinh thái Google" gom nhiều công nghệ khác loại, nên nhóm **chọn một công nghệ chính, rất có thể là AppSheet**, và dùng phần còn lại làm hỗ trợ có lý do. Chiều sâu không đến từ số bảng hay số màn hình mà từ **mô hình dữ liệu, biểu thức, phân quyền, tự động hóa và offline/sync**. Nhóm không cần viết code nếu nền tảng giải quyết được bằng cơ chế nguyên bản, nhưng vẫn nộp bộ ca kiểm thử và minh chứng như các nhóm khác.

1. Bắt buộc — nhóm nào cũng phải tìm hiểu và làm được

A. Nền tảng chung của tự động hóa no-code

- **Trigger và luồng tự động**: sự kiện (thêm hoặc sửa dòng), lịch hoặc thao tác của người dùng; vì sao chọn loại này.
- **Rẽ nhánh theo điều kiện** và có ca kiểm thử cho từng nhánh.
- **Biến đổi dữ liệu và xử lý trường thiếu** bằng biểu thức.
- **Xử lý lỗi**: nhận biết khi một bước thất bại, có hành vi xác định (thông báo, ghi nhận) và biết xem lỗi ở đâu.
- **Tích hợp từ hai hệ thống trở lên**: ví dụ Google Sheets/CSDL, email, Looker Studio, Apps Script.

B. Cơ chế riêng của AppSheet

- **Mô hình dữ liệu**: `Key`, `Ref`, quan hệ giữa các bảng; AppSheet dereference dữ liệu thế nào; vẽ sơ đồ bảng và giải thích một quan hệ thật.
- **Expression có ý nghĩa**: do nhóm tự viết cho giá trị tính toán (computed), kiểm tra hợp lệ (valid-if), hiển thị có điều kiện (show-if), lọc; không chỉ dùng biểu thức mặc định.
- **View và Action**: tổ chức giao diện theo dữ liệu; action thay đổi dòng, điều hướng hoặc kích hoạt quy trình ra sao.
- **Quyền truy cập dữ liệu**: người dùng nào xem hoặc sửa được gì; phân biệt ẩn trên giao diện (`Show_If`) với giới hạn dữ liệu thực sự tải về thiết bị.

2. Tự chọn theo ứng dụng — chỉ chọn cái ứng dụng của em thật sự cần

Không có số lượng cứng. Đã chọn mục nào thì phải giải thích được cơ chế, cấu hình và kết quả của mục đó; không chọn thì không cần nhét vào.

Cơ chế	Nên chọn khi	Cần chứng minh
Slice (tập dữ liệu có điều kiện)	Cần hiển thị hoặc giới hạn tập con	Phân biệt Slice với Security filter bằng ví dụ
Bot và Automation (event, process, task)	Có tự động hóa theo sự kiện hoặc lịch	Bot có điều kiện và chạy hai nhánh khác nhau; xem lịch sử chạy
Security filter	Nhiều người dùng, dữ liệu riêng tư	Người dùng A không thấy dữ liệu của B, chứng minh ở dữ liệu tải về chứ không chỉ ở view
Role-based view và action	Nhiều vai trò	Hai vai trò thấy hai bộ view hoặc action khác nhau
Offline và đồng bộ, xử lý xung đột	Dùng ở nơi mất mạng	Sửa khi offline, đồng bộ lại, ca hai người sửa cùng dòng
Validation biểu mẫu, dropdown phụ thuộc	Nhập liệu phức tạp	Ca dữ liệu sai bị chặn; dropdown lọc theo lựa chọn trước
Báo cáo, dashboard, biểu đồ (view hoặc Looker Studio)	Cần thống kê	Số liệu khớp với dữ liệu nguồn
Apps Script hoặc API/webhook	Nền tảng chưa đáp ứng được một nhu cầu	Một hàm có đầu vào, đầu ra, được gọi từ Bot; ca lỗi

3. Nâng cao — tự chọn để tạo chiều sâu

Không có số lượng cứng; nếu chọn, ưu tiên ít mục nhưng làm sâu, có minh chứng và có số liệu khi phù hợp. Một số mục (TLS, phân quyền, tương thích phiên bản…) chứng minh bằng test hoặc trace thay vì số liệu.

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Bảo mật dữ liệu sâu	Security filter kết hợp vai trò; thử truy cập bằng hai tài khoản khác nhau; kiểm tra dữ liệu thật sự tải về (không chỉ hiển thị).

Hướng nâng cao	Làm gì và chứng minh bằng gì (số liệu, test hoặc trace)
Bộ ca kiểm thử có hệ thống	Khoảng 15-20 ca gồm ca lỗi, kết quả đạt hoặc không đạt có ảnh; kiểm thử biểu thức và Bot.
Dữ liệu lớn và hiệu năng	Bảng nhiều nghìn dòng; đo thời gian đồng bộ hoặc tải trước/sau khi dùng Slice và security filter.
Looker Studio kết nối dữ liệu	Nguồn dữ liệu, trường tính toán, bộ lọc, chia sẻ có kiểm soát quyền; số liệu kiểm chứng với nguồn.
Apps Script có kiểm thử	Hàm xử lý có đầu vào và đầu ra rõ, xử lý lỗi và hạn mức (quota); thử ca lỗi.
Quản lý phiên bản	Xuất mẫu ứng dụng hoặc bản sao; hướng dẫn nhóm khác dựng lại.

4. Minh chứng cần có trong báo cáo và repo

- Sơ đồ các bảng với Key và Ref, kèm giải thích một quan hệ thật.
- Ít nhất vài expression do nhóm tự viết, kèm ảnh tình huống đúng và sai.
- Nếu dùng Bot, Security filter hoặc Offline: ca demo cụ thể, không chỉ bật tùy chọn.
- Bảng ca kiểm thử (khoảng 15-20 ca, gồm ca lỗi) và kết quả; một sự cố giả lập kèm cách phát hiện (lịch sử Bot) và sửa.
- Mẫu ứng dụng hoặc tệp dữ liệu và hướng dẫn tạo tài khoản dùng thử để nhóm khác dựng lại trong 30-45 phút.
- Bảng phủ kỹ thuật: mỗi mục đã làm trỏ tới bảng, view hoặc expression cụ thể.

5. Chưa được xem là đạt nếu chỉ…

- Ứng dụng hai bảng CRUD, dùng view và form mặc định, gần như không có expression.
- Dùng `Show_If` để "ẩn nút" rồi coi là phân quyền trong khi dữ liệu vẫn tải về đầy đủ.
- Tạo rất nhiều view nhưng logic dữ liệu, tự động hóa và bảo mật vẫn đơn giản.
- Đăng ký bốn công nghệ (Looker Studio, AppSheet, Firebase, Apps Script) ngang nhau, mỗi cái chỉ ở mức làm quen.

6. Câu hỏi tự kiểm

- `Key` khác `Ref` thế nào? Nếu đổi key thì có hậu quả gì?
- Security filter khác Slice ở điểm nào?
- Bot này được kích hoạt bởi sự kiện nào và nếu task lỗi thì em biết ở đâu?
- Người dùng A không được thấy dữ liệu của B: em chặn ở đâu, và chặn ở view có đủ không?

7. So sánh và phản biện (để viết phần Tổng quan)

Công nghệ thay thế và lý do chọn AppSheet

- **Power Apps + Power Automate**: mạnh hơn về công thức và tích hợp doanh nghiệp, cần giấy phép phù hợp; AppSheet nhanh hơn từ bảng tính có sẵn và hợp hệ Google.
- **Ứng dụng web viết bằng code**: tùy biến hoàn toàn; tốn công hơn cho ứng dụng nội bộ đơn giản.
- **Firebase hoặc Supabase**: là nền tảng backend để tự viết ứng dụng; không phải công cụ no-code. Danh mục dùng chúng làm công nghệ hỗ trợ, không làm công nghệ chính.

Những trường hợp KHÔNG nên dùng AppSheet (nêu ít nhất 2, dựa trên trải nghiệm của nhóm)

- Ứng dụng công khai cần giao diện tùy biến cao hoặc thương hiệu riêng: AppSheet bị giới hạn.
- Dữ liệu rất lớn hoặc cần truy vấn phức tạp: nguồn là bảng tính sẽ chậm; cần CSDL thật.
- Cần kiểm thử tự động và quản lý phiên bản chặt chẽ như mã nguồn: ứng dụng no-code khó làm việc này.

8. Ví dụ bài thực hành tại chỗ

Chỉ là ví dụ. Đề chính thức được ra từ chính đồ án nhóm đã nộp.

- Thêm một `Ref` và dereference vào luồng đang có.
- Thêm security filter hoặc quy tắc vai trò cho một bảng.
- Thêm một Bot có điều kiện và chứng minh hai nhánh khác nhau.